SANDIA REPORT

SAND2016-6848
Unlimited Release
Printed August 2016

The Sandia Matlab AnalysiS Hierarchy (SMASH) toolbox

Daniel H. Dolan, Tommy Ao, and Sean C. Grant

Prepared by
Sandia National Laboratories
Albuquerque, New Mexico 87185 and Livermore, California 94550

Sandia National Laboratories is a multi-program laboratory managed and operated by Sandia Corporation, a wholly owned subsidiary of Lockheed Martin Corporation, for the U.S. Department of Energy's National Nuclear Security Administration under contract DE-AC04-94AL85000.

Approved for public release; further dissemination unlimited.

Sandia National Laboratories

Issued by Sandia National Laboratories, operated for the United States Department of Energy by Sandia Corporation.

NOTICE: This report was prepared as an account of work sponsored by an agency of the United States Government. Neither the United States Government, nor any agency thereof, nor any of their employees, nor any of their contractors, subcontractors, or their employees, make any warranty, express or implied, or assume any legal liability or responsibility for the accuracy, completeness, or usefulness of any information, apparatus, product, or process disclosed, or represent that its use would not infringe privately owned rights. Reference herein to any specific commercial product, process, or service by trade name, trademark, manufacturer, or otherwise, does not necessarily constitute or imply its endorsement, recommendation, or favoring by the United States Government, any agency thereof, or any of their contractors or subcontractors. The views and opinions expressed herein do not necessarily state or reflect those of the United States Government, any agency thereof, or any of their contractors.

Printed in the United States of America. This report has been reproduced directly from the best available copy.

Available to DOE and DOE contractors from
U.S. Department of Energy
Office of Scientific and Technical Information
P.O. Box 62
Oak Ridge, TN 37831

Telephone: (865) 576-8401
Facsimile: (865) 576-5728
E-Mail: reports@adonis.osti.gov
Online ordering: http://www.osti.gov/bridge

Available to the public from
U.S. Department of Commerce
National Technical Information Service
5285 Port Royal Rd
Springfield, VA 22161

Telephone: (800) 553-6847
Facsimile: (703) 605-6900
E-Mail: orders@ntis.fedworld.gov
Online ordering: http://www.ntis.gov/help/ordermethods.asp?loc=7-4-0#online

SAND2016-6848
Unlimited Release
Printed August 2016

The Sandia Matlab AnalysiS Hierarchy (SMASH) toolbox

Daniel H. Dolan, Tommy Ao, and Sean C. Grant
Sandia National Laboratories
P.O. Box 5800
Albuquerque, NM 87185-1189

Abstract

The SMASH (Sandia Matlab AnalysiS Hierarchy) toolbox is a collection of MATLAB code for data analysis. The toolbox contains general purpose functions, custom class definitions, and self-contained programs aimed at the needs of experimental physicists working in pulsed power research. The initial release (version 1.0) supports file access, signal/image analysis, and user interface creation; custom graphics and generic system tools are also provided. Much of the package is object oriented, encouraging users to build new capabilities from established classes. Future releases will continue this goal, expanding capabilities and streamlining application development.

Acknowledgments

Many people have contributed to the development of the SMASH package. Great thanks is directed to our beta testers: Marcus Knudson, Seth Root, Justin Brown, Patrick Knapp, Ryan McBride, David Bliss, and Randy Spaulding.

The SMASH logo was created by Camber Arnhardt.

Contents

Chapter 1: Introduction **11**
1.1 Getting the toolbox . . . 11
1.2 Toolbox overview . . . 12
1.2.1 Packages . . . 14
1.2.2 Programs . . . 14
1.2.3 Getting help . . . 15
1.3 Report organization . . . 16
1.4 Conventions . . . 17

Chapter 2: File access **19**
2.1 Supported formats . . . 19
2.1.1 Text formats . . . 19
2.1.2 Digitizer formats . . . 21
2.1.3 Image formats . . . 22
2.1.4 Special formats . . . 23
2.2 Reading files . . . 24
2.3 Writing files . . . 25
2.4 Sandia Data Archives . . . 26
2.4.1 MATLAB variables . . . 26
2.4.2 External files . . . 27
2.4.3 Archive management . . . 28

Chapter 3: Signal analysis **29**
3.1 `Signal` class . . . 29
3.1.1 Object creation . . . 30
3.1.2 Visualization . . . 33
3.1.3 Grid and data transformations . . . 34
3.1.4 Object analysis . . . 36
3.1.5 Generating new `Signal` objects . . . 36
3.2 `SignalGroup` class . . . 37
3.2.1 Object creation . . . 38
3.2.2 Differences from the `Signal` class . . . 38
3.2.3 Gathering and splitting signals . . . 39
3.3 Saving and recovering signals . . . 39

Chapter 4: Short-time signal analysis **41**

4.1 `ShortTime` class . . . 41
4.1.1 Object creation . . . 41
4.1.2 Defining partitions . . . 43
4.1.3 Local analysis . . . 44
4.2 `STFT` class . . . 45
4.2.1 FFT settings . . . 45
4.2.2 Local analysis . . . 47
4.3 Adaptive analysis . . . 47
4.4 Saving and recovering short-time analysis . . . 48

Chapter 5: Image analysis **51**
5.1 `Image` class . . . 51
5.1.1 Object creation . . . 55
5.1.2 Visualization . . . 55
5.1.3 Grid and data transformations . . . 56
5.1.4 Object analysis . . . 59
5.2 `ImageGroup` class . . . 61
5.2.1 Object creation . . . 61
5.2.2 Differences from the `Image` class . . . 61
5.2.3 Gathering and splitting images . . . 62
5.3 Saving and recovering images . . . 63

Chapter 6: Image corrections **65**
6.1 Film linearization . . . 65
6.1.1 Using the `StepWedge` class . . . 67
6.1.2 Advanced operations . . . 68
6.2 Geometric distortion . . . 71
6.2.1 Using the `DistortionGrid` class . . . 73
6.2.2 Advanced operations . . . 75
6.3 Saving and recovering image corrections . . . 75

Chapter 7: Graphics **77**
7.1 Plot utilities . . . 77
7.1.1 Special figures . . . 78
7.1.2 Exporting figures . . . 79
7.1.3 Special axes . . . 80
7.1.4 Special plot elements . . . 82
7.2 Custom dialog boxes . . . 83
7.2.1 Creating dialogs . . . 84
7.2.2 Control blocks . . . 84
7.2.3 Refining and using dialogs . . . 87
7.3 Object graphic options . . . 88

Chapter 8: Programs **91**
8.1 Data analysis . . . 91

8.1.1 PointVISAR . . . 92
8.1.2 THRIVE . . . 95
8.1.3 SIRHEN . . . 95
8.2 Data synthesis . . . 95
8.2.1 fringen . . . 95
8.2.2 pyrosim . . . 96
8.3 File processing . . . 96
8.3.1 analyzeSinusoid . . . 96
8.3.2 convertLUNA . . . 98
8.3.3 datninja . . . 99

Chapter 9: Summary and future work **103**
9.1 Version 1.0 summary . . . 103
9.2 Future work . . . 104
9.2.1 Packages under development . . . 104
9.2.2 Future programs . . . 104

References **104**

This page intentionally left blank.

List of Figures

1.1 Toolbox directories 13

3.1 `Signal` class example 31

4.1 `ShortTime` class example 42

5.1 Uniform versus nonuniform grids 52
5.2 `Image` class example 54

6.1 Film exposure overview 66
6.2 Step wedge image 66
6.3 Film and inverse transfer curve 66
6.4 Isopoints in a distorted coordinate 71

7.1 Example MATLAB graphics hierarchy 77
7.2 `Figure` class example 78
7.3 Axes inset example 80
7.4 Axes array example 81
7.5 Error bars and variable-color line example 83
7.6 Custom dialog example 85

8.1 PointVISAR main screen 93
8.2 PointVISAR record settings screen 94
8.3 The analyzeSinusoid program 97
8.4 convertLUNA program 98
8.5 Screen shot `datninja` 100
8.6 Example datninja file 101

List of Tables

2.1 Text formats 20

3.1 Key properties of the `Signal` class 30

3.2 Key methods of the `Signal` class 32

4.1 FFT options for the `STFT` class 46

5.1 Key properties of the `Image` class 52
5.2 Key methods of the `Image` class 53

Chapter 1

Introduction

MATLAB[1] is a powerful environment for numerical calculations and visualization. The language's flexibility, integrated graphics, and extensive documentation make it well suited for rapid code development in scientific and engineering research. Additional benefits include platform independence and legacy code support.

Most MATLAB programs are written to solve specific problems, with little or no consideration of reusability. Code reuse, when it occurs, is usually manual copy/paste operations: it is often it easier to create a new program rather than combine existing programs. While acceptable for small projects, this practice limits the capability of an individual user.

The Sandia Matlab AnalysiS Hierarchy (SMASH) toolbox provides an alternative to a "start from scratch" philosophy. Rebuilding features with minor variations is tedious, detracts from the problem of interest, and impedes new capability development. SMASH eases the burdens of program development and provides a framework for sharing MATLAB code. The goal of SMASH is for users to focus on data analysis and new techniques, not recreating existing solutions.

1.1 Getting the toolbox

SMASH runs on MATLAB releases 2013a or later. None of Mathworks toolboxes (Signal Processing, etc.) are needed to use SMASH, though some features in SMASH can use Mathworks toolboxes (such as Parallel Processing) when available. SMASH is compatible with Mac, Linux, and Windows operating systems.

The SMASH toolbox is licensed for government use. Prospective users should contact Daniel Dolan (`dhdolan@sandia`) to initiate a new license or determine if an institutional license is already in place. Licensed users can download the toolbox as a ZIP file or clone the toolbox repository from GitHub (`http://github.com`). The package should be unzipped or cloned to a directory named `SMASHtoolbox`. ZIP downloads are initially simpler, but Git clones allow easier incremental updates (which are common). To learn more about Git, see `http://git-scm.com`.

After the `SMASHtoolbox` directory has been loaded onto a particular computer, it must

be added to the MATLAB path. This can be done with MATLAB's interactive path tool or the `addpath` function. For example:

```
addpath('~/SMASHtoolbox');
```

indicates that the SMASH toolbox is located in the user directory of a Mac or Linux system. Successful SMASH toolbox installation with can be verified by typing:

```
aboutSMASH
```

in the MATLAB command window. If the SMASH toolbox is present and has been property added to the path, the current version number and commit date/time (if available) will be displayed.

1.2 Toolbox overview

The SMASH toolbox is a collection of MATLAB code developed at Sandia National Laboratories. Its capabilities are strongly influenced by the needs of experimentalists in the pulsed power division, particularly dynamic compression and high-energy density physics research.

Figure 1.1 summarizes the core SMASH toolbox directories. These directories contain MATLAB functions, classes, packages, utilities, and programs.

- Functions accept inputs within parentheses and return outputs within square brackets:

  ```
  [...] = function_name(...);
  ```

  The ellipsis are a shorthand for one or more inputs/outputs.

- Classes define custom objects:

  ```
  object=class_name(...);
  ```

  Objects combine information (properties) with relevant operations (methods). Properties may be accessed with dot notation:

  ```
  value=object.property_name;
  object.property_name=value;
  ```

  if permitted by the class (properties may be read/write protected). Object methods are called using function syntax with the object as the first argument.*

  ```
  [...]=method_name(object,...);
  ```

*MATLAB also accepts dot notation for method calls, but this syntax is not used here.

```
/SMASHtoolbox
     /+SMASH (package)
          /+FileAccess (sub package)
               (functions and classes)
          /+ImageAnalysis (sub package)
               (classes)
          /+General (sub package)
               enprint.m
               (functions)
          /+Graphics (sub package)
               (functions)
          /+MUI (sub package)
               (classes)
          /+SignalAnalysis (sub package)
               (classes)
          /+System (sub package)
               (functions)
     /programs
          /convertLUNA
          /datninja
          /fringen
          /PointVISAR
          /pyrosim
          /SIRHEN
          /THRIVE
     aboutSMASH.m
     loadSMASH.m
```

Figure 1.1. Toolbox directories

- Packages are directories that begin with "+" sign, which MATLAB treats differently than standard directories. Package directories may contain functions, classes, and sub-packages. Most of the SMASH toolbox's capabilities are located inside a package.
- Utilities are SMASH toolbox functions that do not reside inside a package, such as `aboutSMASH` and `loadSMASH`.
- Programs are self-contained functions for a particular task, usually within a graphical user interface.

1.2.1 Packages

As shown in Figure 1.1, the SMASH toolbox has a primary package called `SMASH`. Within the primary package are sub-packages named for their application: `SignalAnalysis` supports signal analysis, `ImageAnalysis` supports image analysis, and so forth. Package directories cannot be added to the MATLAB path, but their contents can be accessed in several ways.

Consider the `enprint` function located in the `General` package, which writes a number to string using engineering notation.

- Dot notation combines package name(s) with the function name.

```
[...]=SMASH.General.enprint(...);
```

- The function can be imported from the package into current workspace.

```
import SMASH.General.enprint % import enprint only
import SMASH.General.* % import entire package
```

  Once imported, the function may be called directly.

```
[...]=enprint(...);
```

  NOTE: imports in the base workspace (command window) do not propagate into functions. The MATLAB function `clear all` removes package imports from all workspaces.

For brevity, all examples in this report assume the appropriate package has been imported.

1.2.2 Programs

Toolbox programs are contained in standard directory called `programs`. Each program is stored in sub-directory of the same name, *e.g.* the PointVISAR program resides in the `PointVISAR` directory. Programs are *always* launched by a function file of the same name,

such as `PointVISAR.m`. Program directories can be added to the MATLAB path, but subdirectories below that level (particularly those named `private`) should *not* be added to the path.

The utility function `loadSMASH` provides convenient access to the toolbox programs. For example:

```
loadSMASH -program PointVISAR
PointVISAR
```

adds the `PointVISAR` directory onto the MATLAB path and launches the program. Programs can be loaded individually or all at once.

```
loadSMASH -program * % load all programs, display results
loadSMASH -silent -program * % load all programs, no display
```

Programs can be loaded in a startup file to make them available throughout a MATLAB session.

1.2.3 Getting help

The SMASH toolbox has considerable online documentation. The `help` command shows documentation in the MATLAB command window.

```
help SMASHtoolbox
```

Hypertext links are available in both cases to browse forward and backward through the documentation. Help for specific package or package components can also be accessed directly.

```
help SMASH % SMASH package documentation
help SMASH.SignalAnalysis % SignalAnalysis package documentation
help SMASH.General.enprint % enprint function documentation
```

The `doc` command works in a similar fashion, but documentation is displayed in a separate window.

```
doc SMASHtoolbox % toolbox documentation
doc SMASH % SMASH package documentation
doc SMASH.SignalAnalysis % SignalAnalysis package documentation
doc SMASH.General.enprint % enprint function documentation
```

Hypertext links may generate new tabs in the documentation window for MATLAB releases 2014a and later. SMASH provides a function called `fixDoc` (in the `System` package) to keep documentation within a single tab for easier forward/reverse navigation.

Many of the core classes in SMASH allow users to access documentation directly through the object. For example, suppose that the variable "mydata" is a `Signal` object (from the `SignalAnalysis` class). The command:

```
doc(mydata)
```

will launch the online documentation for the `Signal` class. Recent MATLAB releases (2016a and later) automatically implement this feature for all objects. This convenient shorthand should be used with caution—it may not work on all MATLAB installations.

1.3 Report organization

This report describes the initial SMASH toolbox release, version 1.0. Even within this version, only some aspects of the SMASH toolbox are described in depth. This document focuses on the underlying concepts in the SMASH toolbox, which will persist throughout future releases. Details for any specific feature may be found in the online documentation.

Chapter 2 describes the `FileAccess` package, which helps MATLAB process data files. The package supports a variety of text and binary file formats, some common and others more esoteric. Functions in this package provide direct, low-level access to data files. These functions are used throughout the toolbox to automatically manage data import, archival, and restoration.

Chapter 3 describes the `SignalAnalysis` package. The discussion explores the `Signal` and `SignalGroup` classes, which represent one-dimensional data. These classes support various analysis operations, some of which are described here. Chapter 4 describes a related set of classes for short-time signal analysis.

Chapter 5 describes the `ImageAnalysis` package. The discussion explores the `Image` and `ImageGroup` classes, which represent two-dimensional data. General image analysis operations supported by the classes are summarized. Chapter 6 discusses two additional classes—`StepWedge` and `DistortionGrid`—included with the package for correcting `Image` objects.

Chapter 7 discusses some of the graphical capabilities provided by the SMASH toolbox. Some of these features are stand alone functions for specific tasks, while others support display options for signal/image visualization. A brief introduction to the `MUI` package is given, illustrating the use of custom dialog boxes.

Chapter 8 describes various programs distributed with the SMASH toolbox. Some of these programs significantly predate the SMASH toolbox and do not make any user of the SMASH package; recent programs rely heavily on the package. The programs fall into three categories: data analysis, data synthesis, and file processing.

1.4 Conventions

Several conventions are used throughout this report. Boxed regions of fixed-width text indicate MATLAB code, which could be used in the command window or within scripts and functions.

```
% This is a MATLAB comment
aboutSMASH % MATLAB command followed by a comment
```

Fixed-width names, such as `SMASH`, indicate references to function, class, package, program, directory, and file names within the text.

MATLAB is case sensitive, it is important that users reference the correct spelling of all SMASH toolbox items. SMASH uses camel case for multi-word package, class names, and class properties; acronyms, such as SMASH, are always capitalized. Several examples are given below.

- `SignalAnalysis` (package name)
- `object=Signal(...)` (class name)
- `value=object.Data` (property name)

Many function and method names begin with a lower case letter and use camel case thereafter, as necessary.

- `enprint(...)` (function name)
- `readFile(...)` (function name)
- `show(...)` (method name)

This convention is meant to clearly distinguish methods from properties when dealing with class objects, but is not strictly enforced in older functions, methods, and programs. When in doubt, consult the online documentation.

Most classes provided by the SMASH toolbox are value classes; classes derived from the handle class are explicitly noted. A consequence of this design choice is that methods often return a modified version of the input object as the first output. This output can assigned to a new object or written over the previous object.

```
new=method_name(object,...); % create new object
object=method_name(object,...); % overwrite existing object
```

New objects are distinct from source objects—modifications to one do not propagate to the other. Most methods internally assume that the modified object will overwrite the source object, so there is no penalty for having two object copies in memory at the same time. In this report, examples generally use the replacement approach unless doing so suggests an unintended accumulation of object changes.

Chapter 2

File access

Many MATLAB sessions begin and end with file access. Experimental measurements and computational simulations must often be loaded into MATLAB from a file. After processing and visualization are complete, MATLAB results are written to a file for archival and sharing.

This chapter discuses the capabilities of the `FileAccess` package. Section 2.1 describes the various text and binary formats supported by the package. Section 2.2 shows how these formats are used to read data into MATLAB. Section 2.3 deals with writing data from MATLAB to a file. Special operations for Sandia Data Archive (SDA) files are presented in Section 2.4.

2.1 Supported formats

Table 2.1 lists the file formats recognized by the `FileAccess` package. Each format is associated with one or more file extensions, some familiar (such as *.jpg) and others more esoteric (*.pff). Formats are used to resolve ambiguous file extensions. For example, *.txt, *.dat, *.csv, and *.out files are found in so many different situations it is impossible to determine format from the file extension alone. Similar ambiguities arise in binary files, particularly for *.img and *.wfm extensions. End users, who presumably knows something about the file's origin and contents, must select the appropriate file format when the extension is not unique.

2.1.1 Text formats

There is no consistent relationship between a text file's extension and the underlying organization. Text files commonly use the extensions *.txt, *.dat, *.inp, *.out, or no extension at all. Given the endless variety and easy reconfiguration possible with text, it is difficult (perhaps impossible) to automatically load and interpret a text file. Users must know something about a file's content for data to be loaded from a text file. This knowledge is captured in the formats `'column'`, `'oceanoptics'`, `'optronicslab'`, and`'optronicslabdump'`.

The `'column'` format provides general file support and should be the first choice for

Table 2.1. File formats supported in the FileAccess package. All formats can be used for reading files; only the `'column'`, `'dig'`, and `'sda'` formats can be used for writing files.

Name	File extension(s)	Description
	Text formats	
`'column'`	various	Data stored in columns, optional header
`'oceanoptics'`	various	Ocean Optics spectrometer measurements
`'optronicslab'`	various	OL-750 spectrometer measurements
`'optronicslabdump'`	various	OL-750 spectrometer screen dumps
	Digitizer formats	
`'agilent'`	*.bin, *.h5	Agilent binary signals
`'keysight'`	*.bin, *.h5	Keysight binary signals
`'lecroy'`	*.trc	LeCroy binary signals
`'tektronix'`	*.isf, *.wfm	Tektronix binary signals
`'yokogawa'`	*.wfm	Yokogawa binary signals
`'zdas'`	*.hdf	Z Data Acquisition System signals
`'saturn'`	*.hdf	Saturn Data Acquisition System signals
	Image formats	
`'film'`	*.img[†], *.hdf, *.pff	Microdensitometer film scans
`'graphics'`	*.jpg, *.tif, etc.	Standard graphics formats
`'hamamatsu'`	*.img	Hamamatsu streak camera data
`'optronis'`	*.imd[‡]	Optronis streak camera data
`'plate'`	*.img[⋆], *.tif	Image plate scans (Fuji or Ditabis reader)
`'sbfp'`	*.img	Santa Barbara Focal Plane camera images
`'sydor'`	*.hdf	Sydor streak camera data
`'winspec'`	*.spe	Princeton Instruments WinSpec images
	Special formats	
`'dig'`	*.dig	Nevada Test Site digital signals
`'pff'`	*.pff	Sandia Portable File Format files
`'sda'`	*.sda	Sandia Data Archive files

[†]Must be accompanied by a *.hdr file of the same name
[‡]May be accompanied by an *.imi file of the same name
[⋆]Must be accompanied by an *.inf file of the same name

loading data from a text file. This format applies to any file containing numeric data columns, possibly preceded by a text header. Files may contain an arbitrary number of columns with various delimiters (white space, commas, semicolons, or ampersands). Numeric data after the header is read into MATLAB until the end of the file is reached or the column layout changes; anything after a format change is ignored. The column format can be used for reading from and writing to a text file.

The `'oceanoptics'` format applies to files created by Ocean Optics spectrometers. These files contain a single measurement column with a special header. This header is used to generate the wavelength variable associated with the measurement column. This format is for reading only.

The `'optronicslab'` and `'optronicslabdump'` formats apply to files created by an OL-750 spectrometer. A standard scan (`'optronicslab'`) is stored as a single column for the measurement (possibly averaged over several scans), with information about the wavelength grid in the header. More comprehensive information is found in the screen dumps (`'optronicslabdump'`), which contain measurements as they are obtained, before averaging is applied. These formats are for reading only.

2.1.2 Digitizer formats

Digitizers often use binary files to efficiently store large data sets. These files are strictly organized in a manner that does not frequently change. Several file extensions used by digitizers are ambiguous, so users must often know which type of digitizer the file was created with. The `FileAccess` package supports a variety of digitizer formats: `'agilent'`, `'keysight'`, `'lecroy'`, `'tektronix'`, `'yokogawa'`, `'zdas'`, and `'saturn'`. These formats are for reading files only.

Agilent and Keysight digitizers (`keysight.com`) use multi-record *.bin and *.h5 files. Both file types are supported by the `'agilent'` and `'keysight'` formats.* Records are indexed by creation order and typically correspond to the channels active when the file was saved: index 1 is usually the first digitizer channel, index 2 is usually the second digitizer channel, and so forth. Inactive channels are skipped (for example, index 2 may be the third digitizer channel if second channel was disabled), and digitizer measurements (such as Fourier transforms) are added after the last active channel. Explicitly specifying these formats is typically optional (for now) in SMASH, but doing so is highly recommended.

Lecroy (`teledynelecroy.com`) digitizers use single-record *.trc files. This extension is currently unique in SMASH, so format specification is optional.

Tektronix (`tek.com`) digitizers use single-record *.isf and *.wfm files. The latter is more common, but both file types (including all known variations of *.wfm) are supported by the

*These formats are currently interchangeable. The `'agilent'` format is frozen, while the `'keysight'` format will evolve as necessary to support changes made by the vendor.

'tektronix' format. Explicit format specification is required for *.wfm files and optional for *.isf files.

Yokogawa (tmi.yokogawa.com) digitizers use single-record *.wfm files, which are quite different than Tektronix *.wfm files. Explicit 'yokogawa' format specification is required.

Some Sandia facilities merge many digitizer signals to single *.hdf (HDF4) file. These files are supported by the 'zdas' and 'saturn' formats, which users select based on file origin. Each record is associated has a text label for read access, and explicit format specification is required.

2.1.3 Image formats

Image data is usually stored in binary files to minimize storage requirements. File organization is relatively stable, often in a universally understood layout and widely recognized file extensions. Image formats can sometimes be determined automatically from the file extension, but in ambiguous cases (such as *.img files) the user must specify the format before the file can be read. In some cases, a supplemental file (usually sharing the same base name with a different extension) is needed to properly interpret the image file. The FileAccess package supports a variety of image formats: 'graphics', 'film', 'optronis', 'plate', 'sbfp', and 'winspec'. All of these formats are used for reading only.

The 'graphics' format manages all standard graphic files and is the first choice for reading image data. Common file extensions associated with 'graphics' include *.jpg/*.jpeg, *.tif/*.tiff, and *.png; unless otherwise instructed, SMASH associates standard graphic files with this format. The 'graphics' format is essentially a front end to MATLAB's imread function, and data can be written to a graphic file with the imwrite function.

The 'hamamatsu' format manages images acquired from a Hamamatsu (hamamatsu.com) streak camera. Data is stored in an *.img file; no supplemental file is associated with the image. This format must be explicitly specified to resolve file extension ambiguity.

The 'film' format supports microdensitometer film scans. Scans stored in an *.img file must have a supplemental *.hdr file of the same base name (e.g., "myfile.img" and "myfile.hdr") to import the data; scans stored in a *.hdf or *.pff file are self contained. This format must be explicitly specified to read film scan images.

The 'optronis' format manages Optronis (optronis.com) streak camera images, which have a *.imd extension. This format searches for a supplemental *.imi file created by the Optronis software, but this information is not required to read the image. The extension *.imd is currently unique to the 'optronis' format, so explicit format specification is optional.

The 'plate' format handles image plate scans. Plate scans performed with a Fuji (fujifilm.com) system use an *.img file with an associated *.inf file; both files must be present in the same directory. Scans made with the Ditabis (ditabis.com) system use a

.tif/.tiff extension and no supplemental file. This format must be explicitly specified to resolve file extension ambiguity.

The `'sbfp'` format manages images created by Santa Barbara Focal Plane (`sbir.com`) cameras. This camera produces *.img files with no supplemental file. This format must be explicitly specified to resolve file extension ambiguity.

The `'sydor'` format manages images created by Sydor (`sydortechnologies.com`) streak cameras. These images are stored in self-contained *.hdf files; because of the ambiguity of this extension, the format must be explicitly specified. Sydor camera files actually store two images: a foreground and background measurement. The latter is automatically subtracted from the former when file is loaded.

The `'winspec'` format supports data stored by the WinSpec program, which is usually associated with Princeton Instruments (`princetoninstruments.com`) cameras. Data files always use the *.spe extension, which is unique to this format. As such, explicit format specification is currently optional.

2.1.4 Special formats

The `FileAccess` class supports several formats associated with the national laboratories. The binary files associated with these formats use extensions of the same name, so there is usually no format ambiguity. The `FileAccess` package supports three special formats: `'dig'`, `'pff'`, and `'sda'`.

The `'dig'` format is often used for signal measurements at the Nevada Test Site, but can found at other Department of Energy laboratories. These files contain a single record of digitizer samples with information for reconstructing the sample time base. SMASH can read and write with this format.

The '`pff'` format manages access to Portable File Format files.[2] These files are often associated with QUICKSILVER codes, but are also found throughout the pulsed power community via the PFIDL library.[3] PFF files may contain multiple records of various types—uniformly sampled signal data, image data, and much more—accessed by numeric index. SMASH can read and write with this format with the following caveats.

- Film densitometry scans stored in a *.pff file should be read with the `'film'` format.
- All PFF record types can be read but only one record type (PFTNGD) written. For more information, see Reference 2.

PFF access is intended for legacy application support—file creation with this format is strongly discouraged. This format cannot be used by the `writeFile` function, but some SMASH classes (notably Signal and Image) have an export method for writing data to a *.pff file.

The 'sda' format provides access to *.sda (Sandia Data Archive) files, which are an integral part of the SMASH toolbox. Reference 4 discusses the motivation and layout of *.sda files. This format can be used for reading and writing of nearly any type of data supported by MATLAB. Section 2.4 describes additional capabilities of SDA files not found in other formats.

2.2 Reading files

The `FileAccess` package provides the `readFile` function to read information from a file into MATLAB.

```
data=readFile(filename,format,record);
```

If no file name or an empty file name is passed, the user is prompted to select an existing file. When the format can be determined from the file's extension, the second input is optional. Extensions associated with more than one format require user guidance if the second input is not used. The output structure `data` contains base and format-specific fields.

The third input (`record`) in the `readFile` function is relevant only to file formats that support multiple records.

- Specific columns in a text file can be requested with the record input.
- Individual signals in Agilent/Keysight files are loaded by numeric index. This index *usually* corresponds to the digitizer channel (channel 1 is often "1", etc.) but exceptions are not uncommon.
- Individual signals in ZDAS/Saturn files are loaded with a text label.
- PFF records are loaded by numeric index corresponding to insertion order.
- SDA records are loaded by text label.

Several examples are given below.

```
data=readFile('name.txt','column'); % all data columns
data=readFile('name.txt','column',[1 3]); % columns 1 and 3 only
data=readFile('name.h5','agilent',2); % record 2
data=readFile('name.sda','channel 2'); % record labeled "channel 2"
```

The `probeFile` function reveals the contents of files where multiple records may be present.

```
probeFile(filename,format);
report=probeFile(filename,format);
```

The first example displays file contents in the MATLAB command window. The second example returns the file contents as a structure.

2.3 Writing files

The `FileAccess` package provides the `writeFile` function for writing MATLAB data to a file. This function supports three formats: `'column'`, `'dig'`, and `'sda'`. The `'column'` format is automatically selected unless the requested file name contains the extensions *.dig or *.sda. The general calling syntax is:

```
writeFile(filename,...);
```

where at least one additional input argument is needed.

Writing a numeric table to a file is illustrated below.

```
writeFile(filename,table,[layout],[header]);
```

The first two inputs are mandatory: users must indicate where and what data should be written. The third input `layout` is an optional specification for column printing; the value must be a valid format string for MATLAB's `fprintf` function. The fourth input is an optional cell array for printing a header lines at the beginning of the file. The following examples writes 100 points of sine function to a text file.

```
x=linspace(0,2*pi,100);
y=sin(x);
table=[x(:) y(:)];
writeFile('example.txt',table,...
   '%.3f %.3f',{'Example data file'});
```

The text file begins with a single header line followed by two columns separated with a space. Each column in the file is a floating point number with three decimal places.

Time/signal data may be written to a *.dig file by changing the extension and passing two one-dimensional arrays. Using variables from the previous example:

```
writeFile('example.dig',x,y);
```

No layout specifications or header information may be passed to the file.

MATLAB variables can be written as a record in *.sda file. The general calling syntax is:

```
writeFile(filename,label,data,[description],[deflate]);
```

The first three inputs are mandatory: in addition to the file name and data variable, a unique text label is needed to place data in the archive. An optional text description can be specified to document the record; the label is usually a terse summary of the record, while the description is meant to be more verbose.[4] A lossless compression factor `deflate` (integer 0–9) may also be specified; the default value is 0 (no compression). Continuing the previous examples:

```
data=struct('x',x,'y',y);
writeFile('example.sda','my table',data,...
   'This is a very short description',2)
```

File size decreases with deflate value, though the amount varies greatly with the type of information stored. Large deflate values slow down the write process as MATLAB works harder to compress the file. Reading a compressed record takes essentially the same amount of time as an uncompressed record.

Data can be written to an existing file in one of three ways.

- For single-record files (column or DIG), the user is prompted to overwrite the existing file with new data.
- For multi-record files (SDA), records are added to the archive if the label does not conflict with an existing record.
- File overwrites can be forced by adding the input '-overwrite' at any location after the file name.

  ```
  writeFile(filename,...,'-overwrite',...);
  ```

  This option deletes the existing file (if present) before new data is written. The user is not prompted to confirm the overwrite.

2.4 Sandia Data Archives

SDA files are based on version 5 of the Hierarchical Data Format (HDF5[5]). The format provides multi-record storage that is easily shared between SMASH users. Using file specifications from Reference 4, *.sda files can be read by virtually any computer language. This section describes SDA capabilities provided by the `FileAccess` package.

2.4.1 MATLAB variables

Almost any MATLAB variable can be stored as a record in a SDA file. Supported variable types include numeric arrays, logical arrays, character arrays, cell arrays, structure

arrays, and custom objects.* Custom object storage is a particularly important feature for the SMASH toolbox.

```
% object creation and manipulation
% ...
writeFile('mydata.sda','result',object,[description],[deflate]);
previous=readFile('mydata.sda','sda','result');
```

The first command writes a MATLAB object to the record `'result'` in the file `'mydata.sda'`. The second command shows how this record is read back into MATLAB as the object `previous`. The restored object retains all properties and methods of the original variable for most SMASH classes (`Signal`, `Image`, etc.). If the object class does not support SDA restoration, stored properties are returned as fields of a structure.

The `FileAccess` package functions offer an alternative to MATLAB's built-in `save` and `load` functions, which store/recover entire workspaces (including variable names). Workspaces are saved in a proprietary *.mat file, the format of which has changed over time. It is difficult to identify the information stored in a *.mat file without loading the entire contents, and even then only the variable names/classes are available (no labels or descriptions are provided). While `save` and `load` are adequate for immediate personal use, `writeFile` and `readFile` are recommended for long term storage or data distribution.

2.4.2 External files

Text and binary files can be stored inside a SDA file, allowing support files (such as spreadsheets) and documentation to be kept with user data. A target file is archived into a *.sda file with the `archiveFile` function

```
archiveFile(archive,target,description,deflate);
```

Archived files are automatically labeled with the source file name; an optional description can be specified to elaborate on the file's contents. Lossless compression level may be specified with the optional deflate value (0–9).

Archived files are reconstructed from the *.sda file using the `extractFile` function.

```
extractFile(archive,target,[newfile]);
```

Normally the reconstructed file uses the same name as the record label, which matches the original file name. Extractions can be directed to a different file with the optional third input.

*Reference 4 describes SDA version 1.0, which supports numeric/logical/character/cell arrays as well as scalar structures and custom objects. SMASH currently uses SDA version 1.1, which adds support for structure/object arrays, complex numbers, and sparse matrices.

2.4.3 Archive management

SDA files can be split into smaller SDA files for transmission purposes.

```
splitFile(archive,dsize);
```

Split files use the same base name as the original archive: `mydata.sda` is divided into `mydata_split1.sda`, `mydata_split2.sda`, and so forth.

The number of file splits is determined by the `dsize` input, which determines the maximum number of bytes written to a single dataset in each split file; note that the actual file size is slightly larger due to HDF5 overhead. Maximum dataset size can be specified as an integer number of bytes or a string with kilobyte/megabyte/gigabyte units (KB/MB/GB).

```
splitFile('mydata.sda',1024) % 1024 bytes
splitFile('mydata.sda','1 KB') % 1 kilobyte
```

The `mergeSplits` function recombines splits files into a complete archive.

```
mergeSplits('mydata_split2.sda');
```

Any one of the split file names may be specified—each split file contains a list of files needed to reconstruct the original archive.

Chapter 3

Signal analysis

Signals describe dependent information on a one-dimensional grid. For example, a continuous relationship $y(x)$ may be approximated by discrete arrays x_n and y_n ($n = 1..N$). The dependent data y_n is defined at grid points x_n. Signal analysis operates on the independent grid and/or the dependent data.

This chapter describes the `SignalAnalysis` package. Section 3.1 discusses the `Signal` class, which manages one-dimensional data on a one-dimensional grid. Section 3.2 discusses the `SignalGroup` class, which can represent multiple signals on a common grid. The two classes share similar capabilities, including visualization and transformation. This discussion places greater emphasis on the `Signal` class, with differences in the `SignalGroup` class noted as necessary. Section 3.3 presents the procedure for saving and recovering `Signal` and `SignalGroup` objects.

3.1 `Signal` class

The `Signal` class represents one-dimensional (scalar) data on a one-dimensional grid. This information is stored as a pair of column vectors ($N \times 1$ arrays) in properties `Data` and `Grid`, respectively. The grid array must be a set of unique real numbers that monotonically increase or decrease; the data array is usually a set of real numbers, though complex values are permitted.* Within these constraints, `Signal` objects can describe any measured or simulated signal.

Table 3.1 displays some of the key properties of the `Signal` class. In addition to the properties, a set of methods that operate on `Signal` objects are available. Methods are special functions that use the object as the first input argument; outputs and additional inputs can be used as allowed by the function. A few of the key methods are listed in Table 3.2, and are briefly described in the following sections. More details, as well as other methods, are presented in the online documentation of the SMASH toolbox.

Figure 3.1 illustrates a basic application of the `Signal` class. Additional details may be

*Some methods, notably `view`, may behave strangely for complex data. Complex signals can also be represented as a pair of real signals using the `SignalGroup` class.

Table 3.1. Key properties of the `Signal` class

Property	Description	Default value
`Data`*	Numerical data storage	`[]`
`DataLabel`	Text label for data	`'Data'`
`GraphicOptions`	Graphics object	`'SMASH.General.GraphicOptions'`
`Grid`*	Numerical grid storage	`[]`
`GridLabel`	Text label for grid	`'Grid'`
`LimitIndex`*	Numerical or text range of grid	`'all'`
`Name`	Object name	`'Signal object'`

* protected property—can't be changed directly

found in the following subsections.

3.1.1 Object creation

`Signal` objects may be created directly from existing MATLAB arrays.

```
object=Signal(x,y);
```

The inputs `x` and `y` are the grid and data arrays, respectively. Both inputs must be numeric. Regardless of size, inputs are converted to column arrays in the properties `Grid` and `Data`. Usually the two inputs have the same number of elements (N), but the following exceptions are permitted.

- If the first input is empty (`[]`), the grid array is a series of integers from 1 to N.
- If the first input is a scalar (dx), the grid array is a set of N equally spaced numbers starting from zero with spacing dx.

`Signal` objects can also be created from from data stored in a file.

```
object=Signal(filename,format,record);
```

The inputs are internally passed to the `readFile` function in the `FileAccess` package (Section 2.2).

- If no inputs are given or the first input is empty (`''`), the user will be prompted to select a file.

```
object=Signal([],0:10);
object.GridLabel='Time';
object.DataLabel='Value';
object.GraphicOptions.LineColor='r';
object.GraphicOptions.Title='Here is a red line';
view(object);
```

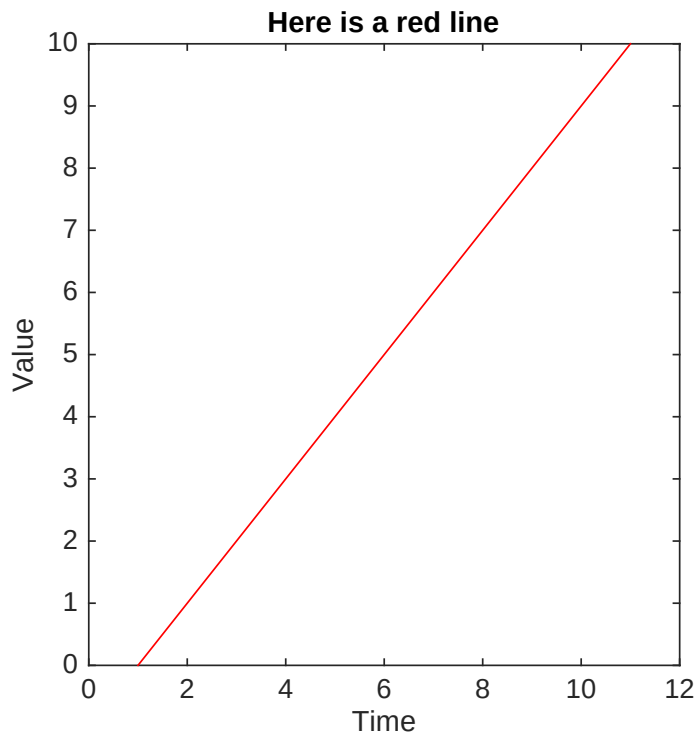

Figure 3.1. `Signal` class example. The code shows an object created directly from a MATLAB array, with some grid/data labelling and display customization.

Table 3.2. Key methods of the `Signal` class

Method	Description
`crop`	Remove data outside specified grid range
`differentiate`	Numerical differentiation of data
`fft`	Fourier transform of data
`integrate`	Numerical integration of data
`limit`	Ignore data outside specified grid range
`locate`	Find specified feature within data
`lookup`	Estimate data value at specific grid location
`map`	Transform grid/data by functionally varying values
`scale`	Scale grid by constant value
`regrid`	Interpolate data to new grid
`replace`	Replace specific grid and data values
`reset`	Reset grid and data arrays
`sharpen`	Subtract local mean from data values
`shift`	Shift grid by constant value
`smooth`	Smooth data using local mean or polynomial
`summarize`	Statistics of data
`view`	Generate plot of `Signal` object

- The second input may be optional if the file extension is unique to a particular format. Users will be prompted to resolved format ambiguity as needed.
- The third input may be optional depending on the file format and contents, which includes single-record digitizer files and two-column text. Record values should be specified for multi-record digitizer files (channel number), PFF files (record number), and SDA files (record label). For text files, the format is a pair of numbers indicating the grid and data columns.

There are several ways to document the contents of a `Signal` object. A general name can be associated with the object.

```
object.Name='Important measurement';
```

This is meant for a terse description of the signal. Extended descriptions can be placed in the `Comments` property via the `comment` method.

```
object=comment(object); % add/edit comments
```

Grid and data labels can be defined for documentation and are used in new plots created by the `view` method.

```
object.GridLabel='Time (s)';
object.DataLabel='Measurement (V)';
view(object);
```

Plot titles can also be specified through the `GraphicOptions` property.

```
object.GraphicOptions.Title='My important measurement';
```

While none of these settings are required, users are strongly encouraged to take advantage of plot labels and name/comments for archived signals.

3.1.2 Visualization

`Signal` objects have a `view` method for visualization. The simplest use of this method:

```
view(object);
```

plots the signal as a line in a new MATLAB figure. The horizontal axis for this plot is determined from the `Signal` object's `Grid` property, and the vertical axis the determined from its `Data` property. The default axes labels for this plot are `'Grid'` and `'Data'`. Custom labels can be specified before the view command is called:

```
object.GridLabel='Time (seconds)';
object.DataLabel='Position (meters)';
view(object);
```

or afterwards using MATLAB's `xlabel` and `ylabel` commands.

`Signal` objects can be plotted to an existing axes by passing the target's graphic handle.

```
view(object,target);
```

The graphic handle for the line created by the `view` method is returned as an output.

```
hline=view(object,...);
```

The `view` method uses settings stored in the `GraphicOptions` property. For example:

```
object.GraphicOptions.LineColor='r';
object.GraphicOptions.Title='Here is a red line';
view(object);
```

plots the `Signal` object as a red line in a new figure with an axes title. Graphic options are controlled by the `GraphicOptions` class (Section 7.3), which enforces MATLAB requirements

for color, line style, and so forth. Line options are always used by the `view` method; options that apply to the axes (or parent figure) are only used when a new figure is created.

3.1.3 Grid and data transformations

The `Grid` and `Data` properties of a `Signal` object are readable but cannot be changed directly.

```
x=object.Grid; % valid
object.Grid=x; % INVALID
```

Simple grid transformations may be performed with the `shift` and `scale` methods.

```
object=shift(object,value); % Grid=Grid+value;
object=scale(object,value); % Grid=Grid*value;
```

The `Data` property can be modified directly by arithmetic operations. For example, the command:

```
new=2*object+1;
```

results in a new object where the `Data` property is twice the original data plus one. In addition to scalar operations, element-wise operations are also permitted on a `Signal` object, between `Signal` objects of the same size, and between a `Signal` object and a numeric array of the same size.

```
new=object.^2; % square signal data
new=object1-object2; % subtract one signal from another
new=object./reference; % divide signal by a reference array
```

More complicated grid and data transformations can be implemented with the `map` method.

```
object=map(object,'Grid',type,argument);
object=map(object,'Data',type,argument);
```

Three map types are supported for converting an input grid/data array to the desired output array.

- `'polynomial'` uses polynomial mapping with a row vector argument.

  ```
  out=polyval(argument,in)
  ```

  The input "argument" should follow MATLAB's descending order convention for polynomials. For example, the mapping $y = x^2 + 2x - 1$ is represented as `[1 2 -1]`.

- `'table'` uses a two-column argument as a lookup table with linear interpolation.

```
out=interp1(argument(argument(:,1),argument(:,2),in,'linear')
```

- `'custom'` uses a user-specified function.

```
out=feval(argument,in);
```

  The argument may be the function's name or a handle to that function (preferred). The function must accept the grid/data array and return an array of the same size.

Shifting, scaling, and mapping a grid preserves the number of points in a signal. The following methods change the number of signal points.

- The `crop` method removes points outside specified grid boundaries.

```
new=crop(object,[xmin xmax]);
```

  Bounds outside the current grid, including `-Inf` and `-Inf`, are truncated to the grid's extreme values. A non-destructive alternative to `crop` is `limit`, which defines a limited region for other methods, such as `view`.

```
object=limit(object,[xmin xmax]);
view(object);
```

  Points outside the grid boundaries are inactive—they are not plotted in this example—but are still part of the object. The active region can be changed to move the active region throughout the signal grid. Limit boundaries are reset to the default state by:

```
object=limit(object,'all');
```

- The `regrid` method accepts a new grid array and interpolates the existing data on that grid.

```
new=regrid(object,x);
```

  If no new grid array is specified, a uniformly spaced array spanning the current grid (using the same number of points) is used.

- The `reset` method may be used to overwrite the grid array.

```
new=reset(object,x,[]); % reset Grid only
new=reset(object,[],y); % reset Data only
new=reset(object,x,y); % reset Grid and Data
```

  The first two calls are permitted if the new array has the same number of points as the existing signal. The third call only requires that changes to the Grid and Data properties be consistent.

Portions of a `Signal` object's data my be overwritten with the `replace` method.

```
new=replace(object,[xmin max],value);
new=replace(object,index,array);
```

The first command replaces all data points inside the specified grid range with a scalar value. The second command uses a logical array to replace portions of the signal data with an array; the replacement array must either be a scalar or have the same number of elements as the total number of true values in the logical index.

3.1.4 Object analysis

The `summarize` method analyzes the the grid and data stored in a `Signal` object.

```
report=summarize(object);
```

Analysis results are returned in a data structure with separate fields for grid and data statistics (mean, standard deviation, etc.).

The `lookup` method estimates data values at requested grid locations.

```
y=lookup(object,x)
```

Linear interpolation is used within the `Signal` object's grid values. Locations outside the grid are assigned NaN.

The `locate` method locates features within a signal by least squares optimization.

```
report=locate(object,'peak'); % locate Gaussian peak
report=locate(object,'step'); % locate erf step
```

Optimization results (fit parameters, etc.) are returned in a data structure. Custom features can be located by passing a handle to a user-defined fit function.

3.1.5 Generating new `Signal` objects

There are a variety of methods for generating new `Signal` objects from an existing object. A few of these methods are summarized here.

- The `differentiate` and `integrate` methods perform numerical differentiation and integration, respectively.

  ```
  new=differentiate(object,parameters);
  new=integrate(object);
  ```

Numerical derivatives are calculated with a local polynomial smoothing,[6] while integration is performed with the trapezoid method. The two methods are not exact inverses of each other!

- The `smooth` and `sharpen` methods apply low-pass and high-pass filtering, respectively.

```
new=smooth(object,'mean',width)
new=sharpen(object,'mean,'width);
```

These examples calculate local means (over the specified width) and either replace the signal with these values (`smooth`) or subtract the local mean from the signal (`sharpen`). Local polynomials[6] and custom kernels can be also be requested.

```
new=smooth(object,'polynomial',[order numpoints]);
new=smooth(object,'kernel',kernel);
```

Adding a smoothed signal to a sharpened signal (with the same `kernel`) recovers the original signal.

- The `fft` method calculates Fourier transforms. The default output is the power spectrum.

```
spectrum=fft(object,...); % single output returns a Signal object
[f,P]=spectrum(object,...); % separate frequency/power arrays
```

The method supports a variety of digital windows, padding options, and spectrum types.

Refer to the online documentation for more details about these methods and a comprehensive list of all current methods.

While `Signal` objects are allowed to have non-uniform grids, several of the above methods require uniform grid spacing. If the object has a non-uniform grid, the user is asked if the object should be transferred to a uniform grid before the method is applied. SMASH ignores uniformity deviations below 0.1%.

3.2 `SignalGroup` class

The `SignalGroup` class represents a set of multiple vector data on a one-dimensional grid. Grid information is stored as a column vector ($N \times 1$ array), while the data is stored as a multi-column table ($N \times M$). As with `Signal` objects, `SignalGroup` objects have a unique, monotonic grid.

3.2.1 Object creation

`SignalGroup` objects are created in a similar fashion as `Signal` objects.

```
object=SignalGroup(x,y);
object=SignalGroup(filename,format,record);
```

A key difference between the classes is that `SignalGroup` supports multiple data arrays, which changes property assignment.

- The size of input `y` is significant when creating an object from numerical arrays. The number of rows N defines the grid size, while the number of columns M defines the number of dependent variables. Flexibility on input `x` remains, subject to size consistency with `y`.
- Multi-column files are usually used to define `SignalGroup` objects. The first column is usually the grid array, though the `record` input can be used to select (or omit) particular columns. `SignalGroup` objects may be constructed from multi-record digitizer files.

`SignalGroup` objects can also be constructed from an existing `Signal` object.

```
object=Signal(...); % construct Signal object
object=SignalGroup(object); % convert Signal to SignalGroup
```

Class conversion is useful for combining individual signals onto a common time base.

3.2.2 Differences from the `Signal` class

`SignalGroup` objects have similar properties as `Signal` objects, but there are some important differences.

- The `Data` property may have multiple columns in a `SignalGroup`. An additional property, `NumberSignals`, indicates the number of signals (and data columns) in the object.
- `SignalGroup` objects have a `Legend` property, which is a cell array of strings. This array is used by the `view` method for plot labeling.

Most `Signal` methods are supported in `SignalGroup` objects. Grid transformations, such as `scale/shift`, work exactly as described above. `Data` transformations (`map`, `smooth`, etc.) are performed internally one signal at a time. Significantly different methods are described below.

- The second input of the `view` method indicates which signals are to be displayed; by default, all signals as plotted.

```
view(object,index); % create new plot
view(object,index,target); % plot in a target axes
h=view(...); % return line handles
```

  All graphic options (line style, etc.) defined in a `SignalGroup` object are used in the plot *except* line color. Line color is automatically assigned using MATLAB's `lines` function, ensuring that each signal is assigned a unique color.

- The `fft` method is disabled for `SignalGroup` objects.

- `SignalGroup` objects have an `evaluate` method that creates a new `SignalGroup` object from a set of user-defined functions. For example, the following command calculates the mean and standard deviation across an existing set of signals.

```
new=evaluate(object,@(x) mean(x,2), @(x) std(x,[],2);
```

3.2.3 Gathering and splitting signals

`SignalGroup` objects can be gathered together with existing `Signal` and `SignalGroup` objects.

```
new=gather(object1,object2,...);
```

The resulting object has a grid spanning all source grid values using the smallest source grid spacing. Missing signal values are assigned to NaN in the output `Data` property.

The `split` method has the opposite effect as `gather`.

```
[obj1,obj2,...]=split(object);
```

Each signal in the group is split into a separate `Signal` object: the first output is the first signal, the second output is the second signal, and so forth. Individual objects can be explicitly specified (as above), or signals can be split automatically into a cell array of `Signal` objects.

```
new=cell(1,object.NumberSignals);
[new{:}]=split(object);
```

3.3 Saving and recovering signals

`Signal` and `SignalGroup` objects may be saved to a file for later use and distribution. The best way of doing this is through functions in the `FileAccess` package (Section 2.4).

Signal data can also be exported to a binary/text file.

The `writeFile` function inserts a `Signal`/`SignalGroup` object as a record in *.sda file.

```
writeFile(filename,record,object,description,deflate);
```

The first three inputs are required; the description and deflate inputs are optional. Record labels should be as short as possible while being unique—using a label already present in the file will generate an error—while descriptions can be as long and detailed as desired. Refer to Section 2.4.1 for more information about storing MATLAB variables in *.sda files.

The `readFile` function extracts records from a *.sda file.

```
previous=readFile(filename,'*.sda',record);
```

`Signal` and `SignalGroup` objects extracted from an archive file are complete replicas of the original object: all properties are precisely the same.

Signal data can be exported to other file formats.

```
export(object,filename);
```

The `export` method supports two formats. If the filename has the extension *.pff, signal data is exported to a Portable File Format file.[2] All other extensions are treated as text files where the `Signal` object's `Grid` and `Data` properties written as columns.

NOTE: signal exports do not preserve all object data! `Grid` and `Data` properties are always present, but other information (labels, graphic options, etc.) are lost. New `Signal` and `SignalGroup` objects can be created from an export file, but doing so is inferior to the `writeFile`/`readFile` workflow described above. Exporting should be reserved for distribution to users/applications that cannot access *.sda files directly.

Chapter 4

Short-time signal analysis

Short-time analysis encompasses a broad range of local signal analysis. Unlike convolution, where a new signal is generated by weighted summation of nearby values, short-time analysis can utilize local time/signal values in any imaginable manner. This general capability is typically slower than convolution but considerably more flexible.

This chapter describes two types of short-time analysis provided by the `SignalAnalysis` package. The `ShortTime` class supports generic short-time analysis, while the `STFT` class adds a frequency spectrum information to the analysis. The first two sections describe these classes from a fixed-duration standpoint. The last section discusses how adaptive analysis can be used in either class.

4.1 `ShortTime` class

The `ShortTime` class does not derive from the `Signal` class—it is not a subclass. Instead, `ShortTime` objects store a `Signal` object within the `Measurement` property, and properties/methods of the stored object are accessible through this property. `ShortTime` objects have additional properties, such as `Partition`, specifically related to short-time analysis.

Figure 4.1 illustrates a basic application of the `ShortTime` class. Additional details may be found in the following subsections.

4.1.1 Object creation

`ShortTime` objects are created in the same manner as `Signal` objects.

```
st=ShortTime(t,s); % existing data arrays
st=ShortTime(filename,format,record); % data from a file
st=ShortTime(object); % convert Signal object
```

The `GridLabel` and `DataLabel` property are automatically set to "Time" and "Signal" at object creation, and the underlying data is mapped onto a uniform grid. Existing `Signal` objects can also be passed as inputs:

```
t=0:(1/10):100;
s=(1+5*(t/max(t))).*randn(size(t));
object=ShortTime(t,s);
object.Measurement.GraphicOptions.LineColor='r';
view(object);
object=partition(object,'Duration',[5 1]);
result=analyze(object,@(local) [std(local.Data) median(local.Data)]);
h=view(result,[],gca);
set(h,'Color','k');
set(h(1),'LineStyle','--');
result.Legend={'Standard deviation','Median'};
legend(h,result.Legend,'Location','northwest');
```

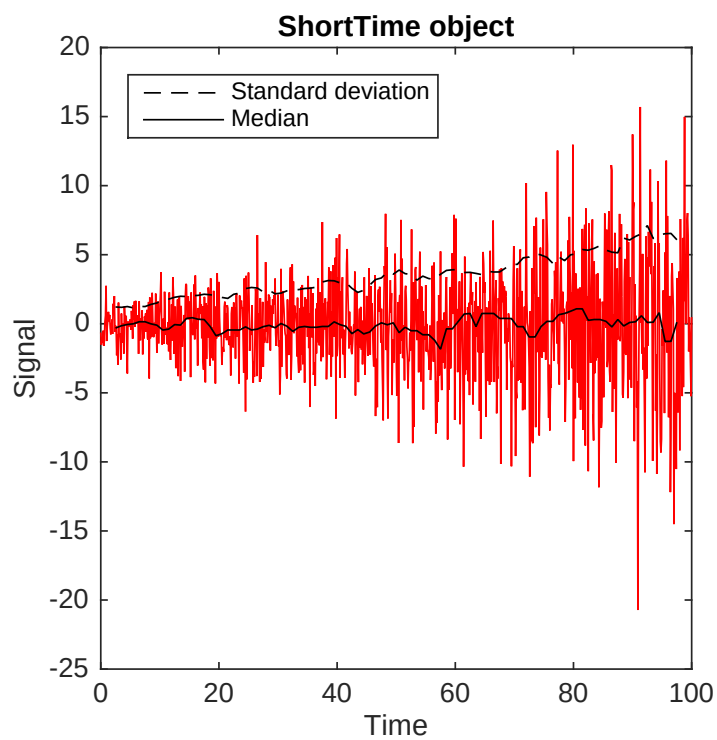

Figure 4.1. `ShortTime` class example

```
st=ShortTime(object); % convert existing signal
```

to convert the information into a `ShortTime` object.

4.1.2 Defining partitions

Partitioning defines the size and location of the local regions used in short-time analysis. These partitions may be specified in several ways.

Point partitioning defines the number of points in each local region. An associated skip value indicates the spacing between subsequent regions.

```
st=partition(st,'Points',points); % skip equals points
st=partition(st,'Points',[points skip]);
```

All partition commands are internally represented in terms of points and skip, explicitly tying partitioning to the discrete time base.

Duration partitioning defines the time span of each local region. An associated advance value, also in time units, indicates spacing between subsequent regions.

```
st=partition(st,'Duration',duration); % advance equals duration
st=partition(st,'Duration',[duration advance]);
```

Duration partitioning is physically intuitive. The inputs "duration" and "advance" use the same units as the underlying time base, so the implications of a particular setting are more obvious.

Block partitioning defines a fixed number of partition blocks, which may (or may not) overlap with one another.

```
st=partition(st,'Blocks',blocks); % skip is zero
st=partition(st,'Blocks',[blocks skip])
```

Block partitioning is a convenient and intrinsically safe way of dividing a signal into local domains. Unlike other approaches, block partition cannot lead to an unexpectedly large number of iterations.

Each partition specification affects the others. For example, specifying the number of analysis blocks defines the number of points within (and time duration of) each analysis.

Current partition settings are stored as a structure stored in the `Partition` property.

```
status=st.Partition;
```

Calling the `partition` method with no output summarizes partition settings in the command window.

```
partition(st);
```

Calling `partition` with an output but no settings invokes the default state.

```
st=partition(st); % 10 blocks with zero overlap
```

Partitioning should be updated whenever changes are made to the signal time base. This is particularly important for block partitioning: cropping/limiting the time range does not automatically update block sizing to the requested number of blocks. Time (`Grid`) unit changes also require updated `partition` calls to accurately reflect the desired partition.

4.1.3 Local analysis

The `analyze` method passes measurement partitions from a `ShortTime` object to a target function and collects the output.

```
result=analyze(st,target);
```

The target function must accept a `Signal` object and return a numeric output. Target functions may be defined anonymously or through a function handle.

```
result=analyze(st,@(local) std(local.Data)); % anonymous function
result=analyze(st,@myfunc); % function handle
```

Target functions that allow multiple arguments must accept calls with a single input, such as `out=myfunc(local)`. Additional inputs must either be optional or managed through an intermediate function handle.

The first output of the local analysis function is used to build a `SignalGroup` object. The time base for this object is based on the center point of each partition. Each signal in the group is built from an array element in the first function output. For example, the first statement generates a one-signal group while the second statement returns a two-signal group.

```
result=analyze(st,@(local) std(local.Data));
result=analyze(st,@(local) [std(local.Data); median(local.Data)]);
```

The number of signals returned by the `analyze` method is determined by the numeric array returned by the target function. The function output array may have any shape (row/column vector, etc.), but the number of elements must remain the same throughout the analysis. These arrays are transferred column-wise into the `SignalGroup` object.

By default, the start and completion of local analysis is indicated in the command window only. A graphical wait bar can be displayed to indicate overall progress.

```
st.ShowWaitbar=true; % show waitbar
st.ShowWaitbar=false; % default state
```

In many cases, routine updates to the wait bar significantly contribute to the overall calculation time. The increment between updates can be adjusted to reduce wait bar overhead while giving the user a sense of how long the calculation will take.

```
st.WaitbarIncrement=0.10; % progress bar updates every 10%
```

The default wait bar increment is 5%, but any value greater than zero and less than one is valid. Optimal values lie in the range 0.25–25%: smaller values are largely invisible to the human eye and larger values diminish the purpose of tracking progress in this fashion.

4.2 `STFT` class

The `STFT` class performs short-time Fourier transform analysis on signal data. This is a subclass of `ShortTime`, so all features described in the previous section apply to `STFT` objects. Objects are created in a similar fashion.

```
mydata=STFT(t,s); % existing data arrays
mydata=STFT(filename,format,record); % data from a file
mydata=STFT(object); % convert Signal/ShortTime object
```

Additional properties and modified methods are described below.

4.2.1 FFT settings

The `FFToptions` property controls how Fourier transforms are calculated in a `STFT` object. Table 4.1 lists the names and default values for these options. Options may be set directly in any order.

```
mydata.FFToptions.Window='hann';
mydata.FFToptions.NumberFrequencies=2000;
```

Invalid option names or values immediately generate error messages.

A comprehensive summary of each FFT option is given below.

Window controls the digital window[7] applied to local signal data *before* the transform is generated. The default value `'hann'` uses a Hann window centered on the local region.

Table 4.1. FFT options for the `STFT` class

Option	Description	Default value
`Window`	Digital window	`'hann'`
`NumberFrequencies`	Min/max number of frequencies	`1000`
`RemoveDC`	Remove DC level	`true`
`FrequencyDomain`	Controls frequency domain	`'positive'`
`SpectrumType`	Controls spectrum type	`'power'`

Other named digital windows include `'hamming'`, `'boxcar'`, and `'gaussian'`. By default, the Gaussian window spans the local region with six (± 3) standard deviations. This convention may be altered by passing a cell array. For example:

```
mydata.FFToptions.Window={'gaussian' 2};
```

spans the local region with four (± 2) standard deviations.

Digital windows may also be specified by a numeric array.

```
mydata.FFToptions.Window=array;
```

This specification does not perform immediate error checking, but severe errors will occur if the size of the window array does not match the local region. This type of window specification is intended for advanced users only.

NumberFrequencies controls the minimum and maximum number of frequencies generated by the Fourier transform. Zero padding is used to generate at least as many transform frequencies as the minimum request. Down sampling is performed, as needed, so that the maximum number of frequency points does not exceed the requested maximum.

This option is usually specified as a single integer, such as 1000 (the default value), that defines the minimum number of frequencies; the maximum number of frequencies is understood to be infinite. Passing a second number allows downsampling to occur. The assigment:

```
mydata.FFToptions.NumberFrequencies=[1000 2000];
```

force the transform to generate no less than 1000 but no more than 2000 frequencies. STFT analysis with downsampling is slower than standard analysis, but the memory requirements are lower.

RemoveDC subtracts the DC (average) value of a local signal prior to the FFT calculation. The default setting is `true`; setting this option to `false` retains the local DC level in the FFT calculation.

FrequencyDomain controls the frequency domain generated by the FFT. The default value `'positive'` generates positive frequencies only. Negative frequency and full frequency transforms can also be requested.

```
mydata.FFToptions.FrequencyDomain='full';
```

SpectrumType controls the type of transform generated by the FFT. The default value `'power'` returns the power spectrum, which is typical for most applications. The complex spectrum may also be requested.

```
mydata.FFToptions.SpectrumType='complex';
```

The complex spectrum carries more information than the power spectrum, but its interpretation is not as visually obvious.

4.2.2 Local analysis

The `analyze` method in the `STFT` class works in a similar fashion as in the `ShortTime` class.

```
result=analyze(mydata,target);
```

The target function must now accept two `Signal` objects: a Fourier transform of the local region and the local region itself. The specified target function must accept these inputs and return a numeric array as the output. For example, this function:

```
function out=myfunc(spectrum,local)
   [value,index]=max(spectrum.Data);
   location=spectrum.Grid(index);
   out=[location; value];
end
result=analyze(mydata,@myfunc);
```

determines the power value and its frequency location. The target function also receives the local signal object, but this information is not used in the example.

As in the `ShortTime` class, `STFT` analysis generates a `SignalGroup` object based on the output from the target function. This output can be a numeric array of any size so long as the number of elements does not change within the current `analyze` session. If no target function is specified, local analysis returns a spectrogram as an `Image` object.

4.3 Adaptive analysis

The `ShortTime` and `STFT` classes are build around fixed partitioning of a `Signal` object: local regions are always drawn with the same number of points. Adaptive analysis is not directly supported, largely because of the difficulty of managing partition changes in a general

framework. Another factor is parallel processing support, which is handled automatically by `ShortTime` and its subclasses. Local analysis is automatically divided amongst multiple MATLAB workers, when present; if not, analysis proceeds in a serial fashion.

Although the `partition` method defines local regions of fixed size, it is possible to perform adaptive analysis on `ShortTime` and `STFT` objects. The time/points between calculations cannot be modified, but the target function can use a subset of the data passed to it. This capability requires some MATLAB sophistication and should be attempted by advanced users only.

Mimicking adaptive analysis begins with an extreme partitioning request.

```
object=partition(object,'Duration',[tmax tstep]);
```

The input "tmax" specifies the maximum duration that the target function will ever use, while "tstep" is the time step. For example, suppose that the local frequency of a chirped signal needs to be measured with optimal time resolution. The maximum duration time would be set to the largest period (lowest frequency) of interest; the time step would be set to be some fraction of the shortest period (highest frequency). During each analysis step, the target function would be passed the local signal (and possible the power spectrum) spanning the longest duration. Intermediate processing within the target function would decide if the local region should be trimmed to 1–2× of the local period and reanalyzed. Such flexibility allows the analysis to benefit from shorter time durations as allowed by the underlying signal.

Adaptive analysis is not without risks. Uncertainty analysis strongly depends on the signal duration, so care must be taken in comparing results where local duration may be different. At minimum, adaptive functions should return the local duration as part of their output array to aid comparisons in different regions.

4.4 Saving and recovering short-time analysis

The `writeFile` function inserts a `ShortTime`/`STFT` object as a record into a *.sda file.

```
writeFile(filename,record,object,description,deflate);
```

The first three inputs are required; the description and deflate inputs are optional. Record labels should be as short as possible while being unique—using a label already present in the file will generate an error—while descriptions can be as long and detailed as desired. Refer to Section 2.4.1 for more information about storing MATLAB variables in *.sda files.

The `readFile` function extracts records from a *.sda file.

```
previous=readFile(filename,'*.sda',record);
```

`ShortTime` and `STFT` objects extracted from an archive file are complete replicas of the original object: all properties are precisely the same.

This page intentionally left blank.

Chapter 5

Image analysis

Images describe dependent information on a two-dimensional grid. For example, a continuous relationship $z(x, y)$ may be approximated by discrete arrays $z_{n,m}$, x_n and y_m ($n = 1..N, m = 1..M$). The dependent data $z_{n,m}$ is defined at independent grid points x_n and y_m. Image analysis is operations on the independent grids and/or the dependent data.

This chapter describes the `ImageAnalysis` package. Section 5.1 discusses the `Image` class, which manages two-dimensional data on two one-dimensional grids. Section 5.2 discusses the `ImageGroup` class, which can represent a set of multiple two-dimensional data on two one-dimensional grids. The two classes share similar capabilities, including visualization and transformations. This discussion places greater emphasis on the `Image` class, with differences in the `ImageGroup` class noted as necessary. Section 5.3 presents the procedure for saving and recovering `Image` and `ImageGroup` objects.

5.1 `Image` class

The `Image` class represents two-dimensional information with a set of two orthogonal, one-dimensional grid arrays. Figure 5.1 illustrates two conceptual grids (x and y) that could be used in an `Image` object. Uniform grids (example A) are most common and nearly all experimental measurements start out on a uniform grid; non-uniform grids (example B) are less common but are supported to handle variable transformations. Non-uniform spacing in one coordinate is assumed to be consistent along the other, as shown in example B, which is to say the grids are parallel. Data not organized in this manner cannot be loaded directly into an `Image` object, but must first pass through MATLAB's `griddata` function using a set of parallel grids.

Table 5.1 displays key properties of the `Image` class. A few of the key methods for `Image` objects are listed in Table 5.2, and are briefly described in this chapter. More details, as well as other methods, are presented in the online documentation for the toolbox.

Figure 5.2 illustrates a basic application of the `Image` class. Additional details may be found in the following subsections.

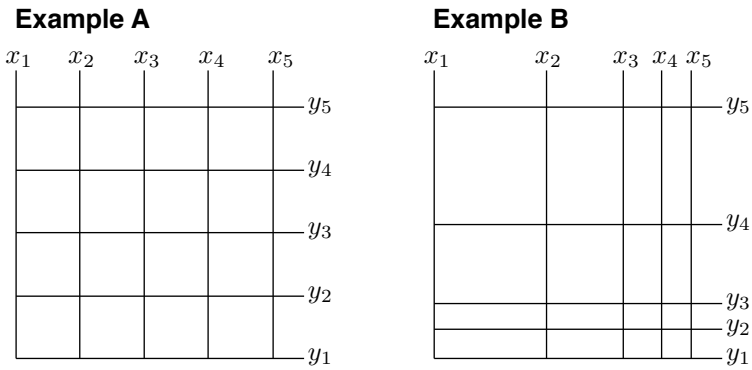

Figure 5.1. Examples of uniform (A) and non-uniform (B) grids

Table 5.1. Key properties of the `Image` class

Property	Description	Default value
`Data`*	Numerical data storage	`[]`
`DataLabel`	Text label for data	`'Data'`
`DataLim`	Numerical or text range of data	`'auto'`
`DataScale`	Numerical scaling for data	`'linear'`
`GraphicOptions`	Graphics options management	`'SMASH.General.GraphicOptions'`
`Grid1`*	Numerical grid1 storage	`[]`
`Grid1Label`	Text label for grid1	`'Grid1'`
`Grid2`*	Numerical grid2 storage	`[]`
`Grid2Label`	Text label for grid2	`'Grid2'`
`Limit1Index`*	Numerical or text range of grid1	`'all'`
`Limit2Index`*	Numerical or text range of grid2	`'all'`
`Name`	Text object name	`'Image object'`

* projected property—can't be changed directly

Table 5.2. Key methods of the `Image` class

Method	Description
`align`	Shift `Image` object using specified grid path
`center`	Adjust grid centers using set of reference points
`crop`	Remove data outside specified grid ranges
`differentiate`	Calculate derivatives of `Image` object
`flip`	Invert `Image` object along one coordinate
`limit`	Confine visualization and analysis of `Image` object to limited region
`lookup`	Find data values at specific grid locations
`map`	Apply variable transformation to grid(s) or data
`mean`	Calculate mean of data
`merge`	Merge multiple `Image` objects into single `Image` object
`regrid`	Interpolate data to new grids
`replace`	Replace specific grid and data values
`reset`	Reset grid and data arrays
`rotate`	Rotate `Image` object by specified direction or angle
`scale`	Scale grid(s) by constant value
`slice`	Generate line outs of `Image` object
`sharpen`	Subtract local mean from data
`shift`	Shift grid(s) by constant value
`smooth`	Smooth data using local mean
`view`	Generate plot of `Image` object

```
x=linspace(-3,3,50);
[x,y]=meshgrid(x,x);
z=exp(-(x.^2+y.^2)/2);
object=Image(x(1,:),y(:,1),z);
view(object,'explore');
```

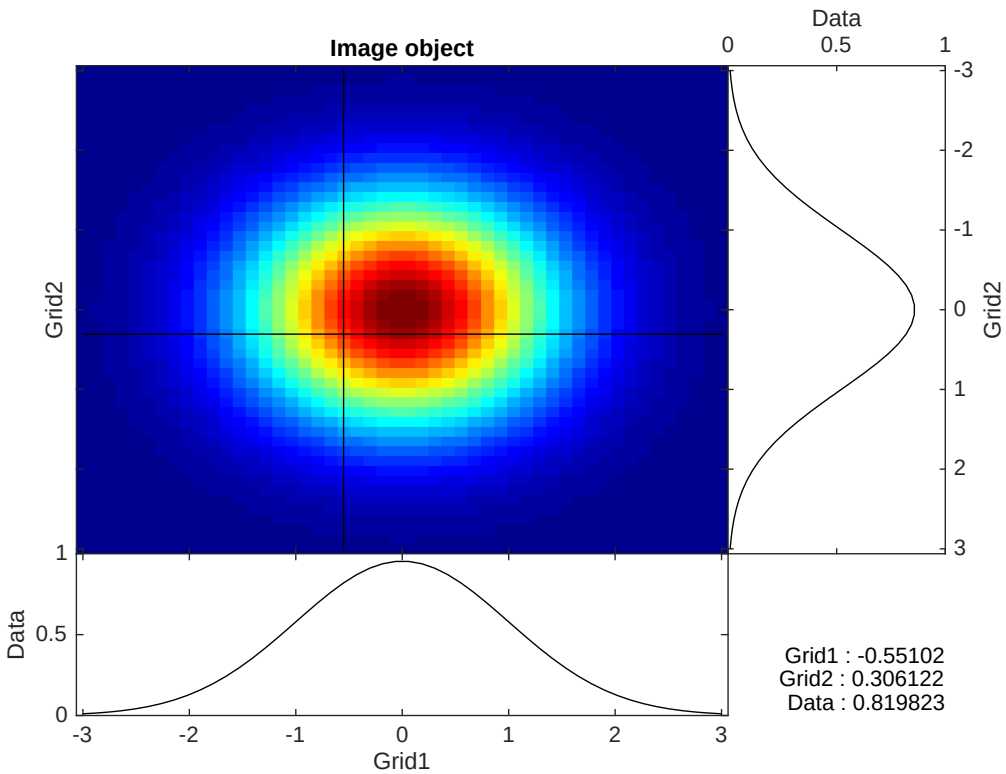

Figure 5.2. `Image` class example

5.1.1 Object creation

When the data of interest already exists as MATLAB arrays `z`, it can be passed directly to the class.

```
object=Image(x,y,z);
```

The grid arrays `x` and `y` are optional; passing empty arrays causes grid arrays to be generated automatically based on the size of `z`.

```
object=Image([],[],z);
```

In most situations, `Image` objects are created by importing information from a data file.

```
object=Image(filename,format,record);
```

For example, passing a character file name (e.g., `'experiment.sda'`) causes the object to be created from an individual file. The second input (`format`) declares how data is accessed from the specified file. `Image` objects can be created from files using the `'film'`, `'graphics'`, `'optronis'`, `'plate'`, `'sbfp'`, and `'winspec'` formats. Images can also be created from `'pff'` and `'sda'` format files, requiring a third input (`record`) to identify a particular dataset within the file. For more information on file formats supported by SMASH, refer to Chapter 2. Multiple images specified with a wild card, such as `'*.imd'`, are automatically summed into a single `Image` object.

5.1.2 Visualization

The `view` method generates graphical displays of an `Image` object. In its most basic form:

```
view(object);
```

displays the `Image` object as a single image plot in a new MATLAB figure. The `detail` view:

```
view(object,'detail');
```

simultaneously displays the full image next to a detailed subregion. The `explore` view:

```
view(object,'explore');
```

shows the image with vertical and horizontal line outs that are updated by clicking on the image or pressing the coordinate keys.

The horizontal x-axis for the plot is determined from the `Image` object's `Grid1` property, the vertical y-axis from its `Grid2` property, and the z-axis from is `Data` property. The

default axes labels for the plot are 'Grid1' and 'Grid2', and 'Data'. Custom labels can be specified before the view command is called.

```
object.Grid1Label='Time (seconds)';
object.Grid2Label='Position (meters)';
object.DataLabel='Intensity (a.u.)';
view(object);
```

The `view` method uses settings stored in the `GraphicOptions` property, which are controlled by the `GraphicOptions` class (Section 7.3). MATLAB automatically displays images with a "golden mean" aspect ratio, which can distort geometric appearances (e.g. squares appear as rectangles, circles as ellipses, etc.). Setting the `AspectRatio` property to `'equal'` overrides this behavior, preserving shape in the displayed image. The `ColorMap` property controls the color map used in the `view` method. The value should be a three-column table of numbers ranging from zero to unity that define the red, green, and blue levels. Use of MATLAB's color map generation functions, such as `'jet'` and `'gray'`, is highly recommended.

The `Image` object's `DataLim` property controls how the data is displayed with respect to the color map. When the value is `'auto'`, the smallest/largest data values are displayed with the lowest/highest color map level. Manual color limits are specified with a two-element array: `[min max]`. The `DataScale` property controls scaling of the data to the color map. In most cases, linear scaling is used. However, data that varies over a large range may be better visualized on a logarithmic (`'log'`) or a decibel (`'dB'`) scale.

5.1.3 Grid and data transformations

The `Grid1` and `Grid2` properties of an `Image` object are readable, but cannot be changed directly. The `scale` and `shift` methods for `Image` objects are analogous to their corresponding methods for `Signal` objects, except a `coordinate` (`'Grid1'` or `'Grid2'`) needs to be specified.

```
object=scale(object,coordinate,value);
object=shift(object,coordinate,value);
```

Similarly, to apply a variable transformation to the grid or data arrays in an `Image` object the `map` method may be used.

```
new=map(object,target,method,value);
```

Three map targets—`'Grid1'`, `'Grid2'`, and `'Data'`—are supported. In each case, the `method` and `value` must be specified.

- `'polynomial'` uses the map value as polynomial array. See MATLAB's `polyval` function for more information.

- `'table'` uses the map value as an interpolation table. This table specifies inputs (first column) that map to outputs (second column). Linear interpolation is used for input values outside of the table's range.
- `'custom'` uses the map value as a function handle. Map inputs are passed to this function for fully customizable transformations.

The `crop` method removes information of the `Image` object outside specified grid ranges. The basic call:

```
new=crop(object);
```

prompts the user to select the "crop" range graphically. The crop range can also be specified directly.

```
new=crop(object,[x1 x2],[y1 y2]);
new=crop(object,[],[y1 y2]); % horizontal limits unchanged
new=crop(object,[x1 x2],[]); % vertical limits unchanged
```

Information outside the crop range is removed from the new object. To preserve this information, the `limit` method should be used.

The `limit` method confines visualization and analysis methods of the `Image` object to a "limited" region.

```
object=limit(object,[x1 x2],[y1 y2]);
object=limit(object,[],[y1 y2]); % horizontal limit unchanged
object=limit(object,[x1 x2],[]); % vertical limit unchanged
```

The `limit` method is conceptually similar to `crop`, but information outside of the limited region is preserved. As such, limits can be applied sequentially and reversibly. To restore the full grid ranges, use string `'all'`.

```
object=limit(object,'all','all');
```

The `regrid` method interpolates an `Image` object onto a new set of grids.

```
new=regrid(object,x,y);
```

Calling this method without new grid values causes the `Image` object to be interpolated onto a set of uniformly spaced grids.

The `reset` method may be used to overwrite the grid and/or data arrays of an `Image` object.

```
new=reset(object,x,y,z); % reset Grid1, Grid2, and Data
```

The `replace` method replaces specified locations of an `Image` object with new values. For example, the data inside the specified bound may be set to a scalar `value`:

```
new=replace(object,[x1 x2],[y1 y2],value);
```

More complicated transformations are also available for `Image` objects. The `align` method shifts an `Image` object such that a specified set of points lie on a straight line.

```
object=align(object,direction,xypath,order);
```

The shift coordinate is specified by `direction` and the shift path is defined by a two-column table `xypath`. A polynomial of specified order is fit through the (x, y) shift path for smoothing and interpolation. Shifting along the `'Grid1'` coordinate causes the smoothed alignment path to be completely vertical; shifting along the `'Grid2'` coordinate causes the alignment path to be completely horizontal.

The `center` method adjusts the grid centers using a set of reference points. By default, the points are assumed to lie on an elliptical boundary.

```
object=center(object);
```

Users are prompted to select boundary points, with the current best fit ellipse shown interactively. Pressing the "Done" button causes the `Image` object's grids to be aligned with the center of the best fit ellipse. Passing a second input overrides the ellipse fit:

```
object=center(object,'points');
```

Centering points are again selected interactively. Pressing "Done" causes the `Image` object to be aligned with the *centroid* of the selected points.

The `flip` method inverts an `Image` object along one coordinate.

```
object=flip(object,'Grid1'); % horizontal inversion
object=flip(object,'Grid2'); % vertical inversion
```

Users are prompted to select a coordinate if the second input is omitted.

The `rotate` method rotates an `Image` object by a direction or specified angle. Directional rotation:

```
new=rotate(object,'left');
new=rotate(object,'right');
```

performs a transpose/flip (`'left'`) or flip/transpose (`'right'`) on the `Image` object's data array. The result is a reversible 90° rotation. To maintain consistency, grid arrays and labels are swapped. The `Image` object may be rotated at a specific angle:

```
new=rotate(object,theta); % degrees
```

which relies on interpolation and is noticeably slower than directional rotation. First, a two-dimensional array (x and y) is created to span the `Image` object's data array. Next, the grid is rotated to a new array (x' and y').

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \tag{5.1}$$

Finally, the original data $z(x, y)$ is linearly interpolated to the rotated data $z'(x', y')$. This information is stored in the output `Image` object's data array, leaving the grid arrays unchanged. Angular rotations are not reversible, and cumulative rotations should be avoided. For example, a single rotation by 45° is recommended instead of a 40° rotation followed by a 5° rotation. Calling the `rotate` method without an second input:

```
new=rotate(object);
```

launches an interactive rotation interface.

5.1.4 Object analysis

There are various data analysis methods available for `Image` objects, and the following section presents only some of the key methods.

The `lookup` method returns data values at specified grid locations.

```
z=lookup(object,x,y);
```

The grid locations `x` and `y` should be scalars, consistently sized arrays, or a scalar/array pair.

The `slice` method creates fixed coordinate curves ("line outs") for an `Image` object.

```
result=slice(object,coordinate,value);
```

The second input defines the fixed slice coordinate (`'Grid1'` or `'Grid2'`); the third output specifies the value(s) of that coordinate. For example:

```
slice(object,'Grid1',[10 20]);
```

creates two slices, in which the dependent data of each slice is an interpolation of the `Image` object's data array at grid1 values 10 and 20. The independent data of each slice is the `Image` object's grid2 array. Calling the `slice` method without an output argument causes the output to displayed in a new figure. If an output argument is used (e.g. `result`), the slices are returned as a new `SignalGroup` object (Section 3.2).

The `mean` method may be used to calculate the average values along a specified coordinate (`'Grid1'` or `'Grid2'`):

```
result=mean(object,coordinate);
```

and generates a new `Signal` object. Note that the averaging is performed over the "limited" region of the object. Alternatively, the `mean` method may be used to find the average scalar value within a region of interest of an `Image` object.

```
value=mean(object,'Region',[xmin xmax],[ymin ymax]);
```

The `merge` method collects multiple `Image` objects into a single `Image` object.

```
new=merge(object1,object2,...);
```

The new `Image` object's grid arrays are determined by the outermost grid points of the source `Image` objects, spaced by the smallest source grid spacings. Source `Image` objects are added to the new grids sequentially. Overlapping information is averaged by the number of source contributions at every point. Points that do not correspond to any source `Image` object are assigned to NaN.

The `smooth` method removes high-frequency variations from an `Image` object.

```
new=smooth(object,choice,value);
```

Two choices—`'mean'` and `'median'`—are provided for standard use. In both cases, the value input controls the smoothing neighborhood. Scalar values are interpreted as symmetric neighborhoods:

```
new=smooth(object,'mean',3); % smooth over 3x3 neighborhood
```

or explicitly by [height width] array.

```
new=smooth(object,'median',[5 3]); % smooth over 5x3 neighborhood
```

Mean smoothing is significantly faster than median smoothing but less robust to outliers. Custom smoothing kernels can also be specified.

```
new=smooth(object,'kernel',kernel);
```

For more information, refer to MATLAB's `conv2` function; the smooth method always uses the `'same'` option of this function.

The `sharpen` method removes low-frequency variations from an `Image` object.

```
new=sharpen(object,choice,value);
```

The `sharpen` method is the complement of the `smooth` method: low-frequency content is calculated by `smooth` and then subtracted from the `Image` object. For more information about the inputs `choice` and `value`, refer to the `smooth` method.

The `differentiate` method performs numerical differentiation along one coordinate (`'Grid1'` or `'Grid2'`).

```
result=differentiate(object,coordinate,[order numpoints]);
```

The last output controls smoothing used in the derivative calculation: a polynomial of specified order is applied locally to specified number of points to remove high-frequency noise (Savitzky-Golay method[6]).

5.2 `ImageGroup` class

The `ImageGroup` class represents a set of multiple two-dimensional data on two one-dimensional grids. Grid information is stored as column vectors ($N \times 1$ and $M \times 1$ arrays), while the data is stored a three-dimensional array ($N \times M \times P$). As with `Image` objects, `ImageGroup` objects have unique, monotonic grids.

5.2.1 Object creation

`ImageGroup` objects are created in a similar fashion as `Image` objects.

```
object=ImageGroup(x,y,z);
object=ImageGroup(filename,format,record);
```

A key difference between the classes is that `ImageGroup` supports multiple data matrices.

`ImageGroup` objects can also be constructed from existing `Image` and/or `ImageGroup` objects.

```
object1=Image(...); % construct Image object
object2=ImageGroup(...); % construct ImageGroup object
object=ImageGroup(object1,object2); % convert objects to single ImageGroup
```

5.2.2 Differences from the `Image` class

`ImageGroup` objects have similar properties as `Image` objects, but there are some important differences.

- The `Data` property may have multiple matrices in an `ImageGroup`. An additional property, `NumberImages`, indicates the number of images in the object.

- An `ImageGroup` object also has a `Legend` property, which is a cell array of strings. This array is used by the `view` method for plot labeling.

Most `Image` methods are supported in `ImageGroup` objects. Simple grid (e.g. `scale`, `shift`, `flip`, `rotate`) and data transformations (e.g. `map`, `smooth`, `sharpen`), work exactly as described above and apply to all images (though the operations are performed internally one image at a time).

In the case of methods with user interfaces (e.g. `crop`, `align`, `center`), a second input (`ImageNumber`) is required to specify the image to be displayed.

```
object=crop(object,ImageNumber);
```

If no `ImageNumber` is specified, the default is to use the first image. After completing the graphical operation, the method is applied to all images within the `ImageGroup` object.

In the `view` method for `ImageGroup` objects, the second input indicates how the images are to be displayed.

- By default, all images are plotted in a `'mosaic'` mode:

```
view(object,'mosaic'); % show all images
```

- To show one image at a time with toggling between images, use the `'snapshot'` mode:

```
view(object,'snapshot'); % selectable images
```

- To show a specific image, input the `ImageNumber`:

```
view(object,5); % show specific image (e.g. 5th image)
```

5.2.3 Gathering and splitting images

`ImageGroup` objects can be gathered together with existing `Image` and `ImageGroup` objects.

```
new=gather(object1,object2,...);
```

The resulting `ImageGroup` object has grid arrays spanning all source grid values using the smallest source grid spacing. Missing data values are assigned to NaN in the output `Data` property.

The `split` method has the opposite effect as `gather`.

```
[obj1,obj2,...]=split(object);
```

By default, all images are output in image order, however specific images can be called if some images are not desired.

```
[img3,img4,img5]=split(object,[3:5]); % output images 3 through 5
```

Each image in the group is split into a separate `Image` object: the first output is the first image called, the second output is the second image called, and so forth. Individual objects can be explicitly specified (as above), or images can be split automatically into a cell array of `Image` objects.

```
new=cell(1,object.NumberImages);
[new{:}]=split(object);
```

5.3 Saving and recovering images

`Image` and `ImageGroup` objects may be saved to a file for later use and distribution. The best way of doing this is through functions in the `FileAccess` package (Section 2.4). Image data can also be exported to a binary/text file.

The `writeFile` function inserts a `Image`/`ImageGroup` object as record into a *.sda file.

```
writeFile(filename,record,object,description,deflate);
```

The first three inputs are required; the description and deflate inputs are optional. Record labels should be as short as possible while being unique— using a label already present in the file will generate an error—while descriptions can be as long and detailed as desired. Refer to Section 2.4.1 for more information about storing MATLAB variables in *.sda files.

The `readFile` function extracts records from a *.sda file.

```
previous=readFile(filename,'sda',record);
```

`Image` and `ImageGroup` objects extracted from an archive file are complete replicas of the original object: all properties are precisely the same.

Image data can be exported to other file formats.

```
export(object,filename);
```

The `export` method supports two formats. If the filename has the extension *.pff, image data is exported to a Portable File Format file.[2] All other extensions are treated as text files where the object's `Grid1`, `Grid2` and `Data` properties written as (`x y z`) columns.

NOTE: image exports do not preserve all object data! `Grid1`, `Grid2`, and `Data` properties are always present, but other information (labels, graphic options, etc.) are lost. New

`Image` and `ImageGroup` objects can be created from an export file, but doing so is inferior to the `writeFile`/`readFile` workflow described above. Exporting should be reserved for distribution to users/applications that cannot access *.sda files directly.

Chapter 6

Image corrections

Measured images often systematically differ from the physical image of interest. Developed photographic films are nonlinear representations of exposure, and geometric distortion may be found in all real imaging systems. Quantitative image analysis can only be performed after these systematic effects are removed.

This chapter describes film and distortion corrections supported in the `ImageAnalysis` package. Film linearization is managed by the `StepWedge` class, while the distortion removal is handled with the `DistortionGrid` class. Both classes rely on a characterization image to determine the appropriate corrections. Analysis from the characterization image are applied to measured images to remove systematic imperfections.

6.1 Film linearization

Consider a region of film exposed to Q photons per unit area (Figure 6.1). After processing, this region has an optical density $D \equiv -\log T$, where T is the optical transmission. The relationship between film density D and exposure Q is monotonic—more photons lead to more darkening reactions—but not linear. Strong exposures cause film saturation at a maximum density, while weak exposures become indistinguishable at the minimum detectable density. The precise link between exposure and density depends on the film type, processing conditions, and densitometer operation.

Suppose a measurement of interest is performed on particular type of photographic film. A second film of the same type can be used to characterize the film transfer function. Passing an exposure Q_0 through a reference density $\tilde{D}$ creates a characterization film exposure $Q_0 10^{-\tilde{D}}$ (Figure 6.1), which corresponds to a processed density D'. A series of reference exposures can be performed simultaneously by placing a step wedge (Figure 6.2) on a reference film with uniform illumination. The measurement and step wedge film are processed simultaneously to account for batch variations and densitometer performance.

The left plot in Figure 6.3 shows example measurements of the forward transfer function. Circles indicate step wedge measurements spanning the film's dynamic range. The vertical axes shows the measured density of the characterization film, while the horizontal axes shows the amount of light passing through the step wedge relative to the median value. Relative

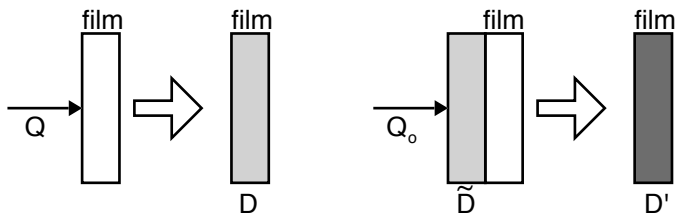

Figure 6.1. (left) Exposure Q leads creates film density D. (right) Exposure Q_0 through a step density $\tilde{D}$ creates reference film density D'.

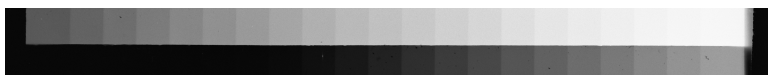

Figure 6.2. Gray scale image of a step wedge. The wedge has 21 density steps (horizontal variation) with additional density placed on half of the image (vertical variation).

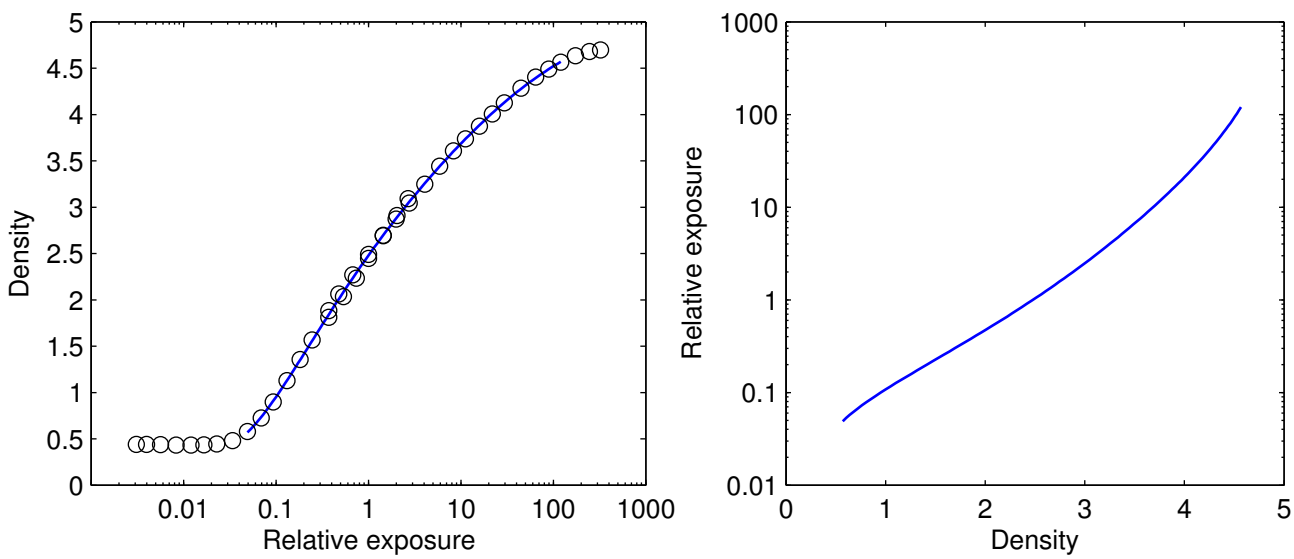

Figure 6.3. Film transfer (left) and inverse transfer (right) curves

exposure eliminates Q_0 (which is often unknown or varies between film batches) from film linearization; separate analysis is needed for absolute calibration. The right plot shows the inverse relationship, which describes how processed film density can be mapped to relative exposure.

Step wedge analysis determines the inverse transfer function using the following steps.

- The logarithm of the relative exposure $x \equiv \tilde{D}_m - \tilde{D}$ is calculated from the known step wedge densities. Reference value $\tilde{D}_m$ is defined by the median wedge value.
- The step wedge image is divided into quasi-constant regions, with boundaries given by sharp changes (large slopes) in the local average. Characterization densities $y \equiv D'$ are determined by the median image value for each step wedge region.
- Measured points (x, y) are divided into low (fog), intermediate, and high (saturation) domains. The intermediate range is defined by a fraction (default value is 95%) of the vertical span. Data in the intermediate domain are fit with a polynomial for smooth interpolation on a fine grid.
- Inverting the interpolated data provides the inverse over a finite range of film density. Densities outside the interpolation range are capped at the minimum/maximum values of the interpolation table.

These operations are implemented in the `StepWedge` class

6.1.1 Using the `StepWedge` class

Step wedge images are associated with film measurements from a particular processing batch. As such, the `StepWedge` class is always used in tandem with the `Image` class. For example:

```
measurement=Image(file1,format,[record]); % data acquired on film
sw=StepWedge(file2,format,[record]); % step wedge
```

creates an `Image` object from one file and `StepWedge` object from another. Measurement files should have the same extension as the step wedge file; the base file names are usually similar. `StepWedge` objects can be stored and recalled from a SDA file using the `FileAccess` package as described in Section 2.4.

```
writeFile(filename,label,sw);
previous_sw=readFile(filename,'sda',label);
```

Image data in a `StepWedge` object is stored as an `Image` object within the `Measurement` property. This data is processed by the `analyze` method to determine the film transfer table.

```
sw=analyze(sw);
```

The analyze method does several things:

- Users are prompted to crop the step wedge image, if it hasn't already been cropped. Regions outside the step wedge must be removed from the image for the next step to be successful.
- Step transitions are identified and used to construct regions of similar processed density.
- The forward and inverse transformations are constructed from the step regions and the reference densities.
- A plot illustrating region selection and forward/inverse transformation is displayed.

Settings for and results from this analysis is stored as structures within the `Settings` and `Results` properties, respectively. These properties cannot be changed directly. Analysis settings can be modified through the `configure` method, but the results are only updated by the `analyze` method. Measurement properties can be modified directly as shown below.

```
object.Measurement.GraphicOptions.ColorMap=gray(64);
object.Measurement.GraphicOptions.Title='My step wedge';
```

The object stored within the `Measurement` property can invoke all capabilities of the Image class (Chapter 5).

The results of a step wedge analysis are transferred to a film measurement by the `apply` method.

```
data=apply(sw,measurement);
```

The output object is a linearized version of the original measurement. This correction should be performed before any other operations: continuous rotation, background corrections, etc.

6.1.2 Advanced operations

The following operations can be used in conjunction with the `analyze` and `apply` methods.

- The `view` method displays the measured image or the analysis results.

  ```
  view(sw,'Measurement');
  view(sw,'Results');
  ```

- The measured image can be interactively/manually cropped to remove all regions outside the step wedge.

```
sw=crop(sw); % interactive cropping
sw=crop(sw,xbound,ybound); % manual cropping
```

  The `analyze` method does not prompt users to crop `StepWedge` objects that have been explicitly cropped. Objects should be reanalyzed whenever the measured image is cropped.

- Measured images must be rotated so that the longest dimensions is aligned with the horizontal axes before.

```
sw=rotate(sw,'auto'); % automatic rotation (90 degree increments)
sw=rotate(sw,'gui'); % interactive rotation
sw=rotate(sw,theta); % manual rotation (counter-clockwise in degrees)
```

  Areas in the rotated image that originate outside of the original image are assigned with NaN values. A different value assignment can be specified by passing a third input argument.

```
sw=rotate(sw,theta,value);
```

- Local artifacts in the step wedge image can be removed with the `clean` method.

```
sw=clean(sw,nhood);
```

  Median filtering over a specified neighborhood ([vertical horizontal]) is used in this method. The technique is very effective if the specified neighborhood is larger than the artifact; unfortunately, is rather slow. Step wedge measurements usually do not requiring cleaning, but this method can help resolve analysis failures.

Step wedge analysis parameters are controlled by the `configure` method. Calling this method without parameters:

```
configure(sw);
```

displays the current value of each parameters. Parameter changes are specified by name/-value pairs.

```
sw=configure(sw,name,value); % change setting
sw=configure(sw,name,[]); % restore default setting
sw=configure(sw,name1,value1,name2,value2,...); % multiple settings
```

Valid parameter names are described below.

StepLevels is an array of reference densities for the step wedge. Step wedges typically have 21 density levels in increments of approximately 0.15. The analyze method uses the number of density values to assist region location, *e.g.* a step with 21 levels should have 20 transitions across the image. The number and value of the StepLevels setting must match the number of steps along the long dimension.

StepOffsets is a array of density offsets along the short dimension of the step wedge. Additional optical density placed on part of the step wedge increases the range of the step wedge, characterizing more of the film's dynamic range. For example, film scans at Sandia use a ≈ 2 density offset on half of a 21 level step wedge to generate 42 levels (as shown in Figure 6.2).

DerivativeParams controls the numerical differentiation used to locate step transitions. This parameter is a pair of integers. The first number indicates the local polynomial order. The second number is the number of points used for the polynomial fit.

HorizontalMargin* specifies the horizontal gap between step transitions and the analysis region. For example, a value of 0.05 indicates that density measurements span the central 90% of each step region.

VerticalMargin* specifies the vertical gap between step transitions and the analysis region.

AnalysisRange* controls the vertical range span of the forward transfer data that is fit with a polynomial.

PolynomialOrder defines the polynomial order used for linearization analysis range.

Normalized parameters are marked with an asterisk; these must be greater than zero and less than unity.

The `analyze` method has two modes. The default mode uses a polynomial fit over the middle of the film's density range.

```
object=analyze(object); % 4th order polynomial fit over middle 95%
object=analyze(object,'middle','Range',[lower upper],'Order',N);
```

Lower/upper range values must be between 0 and 1. Polynomial order must be an integer, and values are typically 2–6; higher values are permitted but may produce unphysical results. An full analysis mode is also provided.

```
object=analyze(object,'full'); % standard settings
object=analyze(object,'full','Order',N,'FogTolerance',tol); % custom settings
```

Full analysis explicitly analyzes the fog, active, and saturation regions.

- The fog domain is a constant density from the lowest exposure, spanning all points within the density tolerance (default 0.05) of the lowest exposure point.

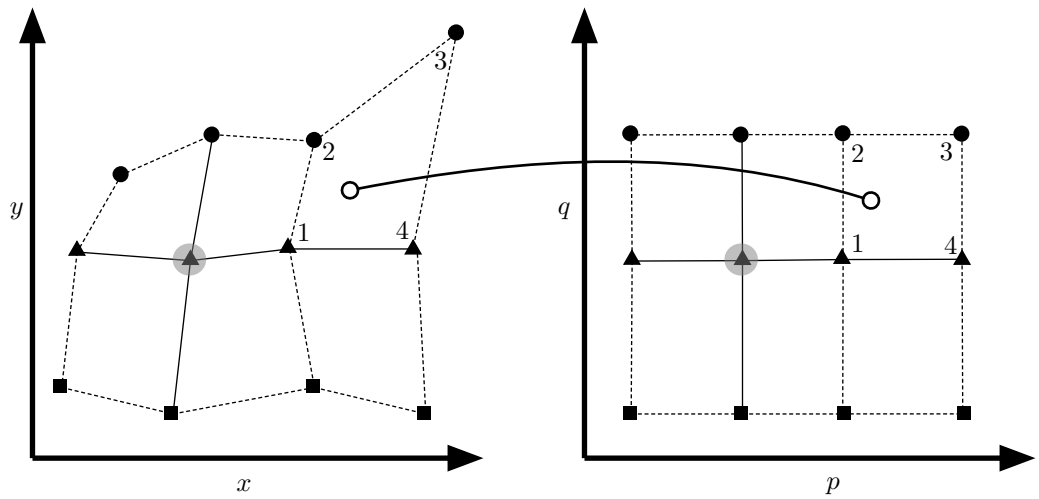

Figure 6.4. Isopoints in a measurement coordinate system (left) transform to a rectangular mesh in the undistorted coordinates (right). The origin of this transformation (shaded marker) is the centermost isopoint. Undistorted mesh spacing is determined by arc length on the central (solid) lines of the distorted coordinates.

- The saturation domain is a constant density at the highest permitted value of the densitometer. This region only exists when measured densities are exactly the same at high exposures.
- The active domain is a monotonic density function between the fog and saturation domains. The slope of the active domain is continuous with the fog domain but not the saturation domain.

The default (middle) analysis is recommended for most situations. Full analysis should only be used when features of interest are very close to the fog/saturation level.

6.2 Geometric distortion

Geometric distortion results from spatial imperfections in a measured image. These imperfections may span the entire image (such as a rotation) or vary throughout the image (such as a shear). In either case, the measured coordinate system systematically differs from the desired coordinate system in some way.

Figure 6.4 illustrates two conceptual coordinate systems. The measurement coordinate system (x, y) has various distortions, while the coordinate system (p, q) is undistorted. These

systems are linked by a set of isopoints that define locations of common (p, q). In this particular example, closed circles indicate a path of fixed q at four values of p. Closed triangles and squares indicate two additional paths of common q at the same values of p. Common p (or q) values between isopoints is absolutely essential, though uniform spacing in p (or q) is not.

Local transformations from (x, y) to (p, q) may be expressed as:

$$p = a + bx + cy + dxy \tag{6.1}$$

$$q = e + fx + gy + hxy \tag{6.2}$$

where parameters a–h are determined by nearest four enclosing isopoints. The open circles in Figure 6.4 are an example of such a transformation using isopoints 1–4. These isopoints define the local translation, scaling, rotation, and shear of the measured coordinate system

$$\underbrace{\begin{bmatrix} p_1 & q_1 \\ p_2 & q_2 \\ p_3 & q_3 \\ p_4 & q_4 \end{bmatrix}}_{\tilde{P}} = \underbrace{\begin{bmatrix} 1 & x_1 & y_1 & x_1y_1 \\ 1 & x_2 & y_2 & x_2y_2 \\ 1 & x_3 & y_3 & x_3y_3 \\ 1 & x_4 & y_4 & x_4y_4 \end{bmatrix}}_{\tilde{P}} \underbrace{\begin{bmatrix} a & e \\ b & f \\ c & g \\ d & h \end{bmatrix}}_{\tilde{M}}$$

$$\tilde{M} = \tilde{X} \backslash \tilde{P} \tag{6.3}$$

Each rectangle in the undistorted coordinate system corresponds to a quadrilateral in measured coordinates and a distinct set of eight transformation parameters.

Geometric distortion removal follows several steps.

- The user selects groups of isopoints in the test image. Each group shares a common value of p (or q), with common values of q (or p) between groups.
- Each isopoint group is sorted, minimizing the distances between adjacent isopoints; ambiguous results are resolved by maximizing the dot product between adjacent points. Groups are sorted orthogonal to the direction of greatest change within the group.
- The centermost isopoint of the centermost group is identified as the transformation origin (shaded circle in Figure 6.4). The p-axis is defined from isopoint median the centermost group; the q-axes is defined by the shortest connection from the origin to the nearest point in adjacent isopoint groups. Solid lines in the figure indicate the curves $q = 0$ and $p = 0$ in the distorted coordinate system.
- Projected isopoints are added to the beginning/ending of each group (as needed) so that all groups contain the same number of points.
- Arc lengths of each isopoint the $q = 0$ and $p = 0$ curves are calculated in the distorted coordinate system. These distances define isopoint locations in the undistorted coordinate system. Consistent vertical/horizontal spacing is applied to all isopoint groups.

- Adjacent connections between isopoints define a set of transformation meshes. The original, distorted mesh is denoted as the isomesh; the transformed, undistorted mesh is called the arcmesh. A particular (x, y) location is mapped to (p, q) by a local transformation:

- Stepping through the arcmesh, regularly spaced set of (p, q) coordinates is transformed to (x, y) coordinates. Interpolation of the measured image on these transformed points removes local distortions

These operations are implemented in the `DistortionGrid` class.

6.2.1 Using the `DistortionGrid` class

Distortion images characterize defects of a specific recording system (optics, readout, etc.). These images are used to correct actual measurements using the same configuration, so `DistortionGrid` objects are always used in tandem with `Image` objects.

```
measurement=Image(file1,format,[record]);
dg=DistortionGrid(file2,format,[record]);
```

For best results, distortion images should use the same file format as the measurement image(s). `DistortionGrid` objects can be stored and recalled from a SDA file using the `FileAccess` package as described in Section 2.4.

```
writeFile(filename,label,dg);
previous_dg=readFile(filename,'sda',label);
```

`DistortionGrid` objects can be constructed from a single image file (as shown above) or from multiple files of the same format. Image summation occurs whenever the specified file name contains a wildcard. The following example merges all Optronis streak camera files into a single distortion grid measurement.

```
dg=DistortionGrid('*.imd','optronis');
```

While this feature is available for all `Image` objects (Section 5.1.1), it is particularly useful for `DistortionGrid` objects. It is often convenient to acquire a group of isopoints in separate image files, changing one coordinate (usually position) between files.

The `locate` method allows the user to specify isopoints in the distortion image. These points may be specified manually with a two-column table; NaN entries indicate separations between isopoint groups.

```
object=locate(object,table); % numeric table
object=locate(object,'table.txt'); % text file containing table
```

Calling the `locate` method without inputs launches interactive selection.

```
object=locate(object);
```

Although interactive selection can be tedious, the human eye is remarkable adept at this task for different configurations (isolated points, line intersections, etc.) and a wide range of signal-noise conditions. `DistortionGrid` objects can be stored in a SDA file, so the effort spent selecting isopoints should not have to repeated.

Isopoint selections is used to define an isomesh in the distorted coordinate system and and undistorted arcmesh (as shown in Figure 6.4). `DistortionGrid` objects store this information properties `IsoPoints` (two-column table with NaN separators), `ArcMesh1`/`ArcMesh2` (rectangular arrays), and `IsoMesh1`/`IsoMesh2` (rectangular arrays). The `view` method displays the distortion image or its associated points/meshes.

```
view(dg); % display image
view(dg,'measurement'); % same as above
view(dg,'isopoints');
view(dg,'isomesh');
view(dg,'arcmesh');
view(dg,'vector');
```

The last example generates a vector field illustrating how points in the distorted image transform to the undistorted coordinate system. By default, each type of visualization is rendered in a new MATLAB figure. Graphics can also be written to an existing axes by passing third input.

```
view(dg,mode,haxes);
```

The `apply` method uses the isomesh and arcmesh to remove distortion from an `Image` object.

```
corrected=apply(dg,measurement);
```

The output "corrected" is a corrected version of the target object, which can be any subclass of the `Image` class. Jacobian corrections are not incorporated in the corrected image, but this information is returned as a second output.

```
[corrected,jacobian]=apply(...);
```

Removing distortion prior to intensity calibration usually makes explicit Jacobian correction unnecessary.

6.2.2 Advanced operations

The `DistortionGrid` class provides several ways of altering the measured image. The `crop`, `rotate`, and `center` methods act on the `Measurement` property in the same manner as the `Image` class methods described in Chapter 5. The `clean` method performs median filtering on the measured image.

```
dg=clean(dg,nhood);
```

The input "nhood" is 1–2 integer values that define the local smoothing neighborhood. This type of smoothing is not strictly needed but can improve visibility by removing extremely bright pixels from the distortion image.

The `blur` method removes sharp variations from the isomesh. Local blurring:

```
dg=blur(dg,'local')
```

replaces each internal mesh point by the average of its neighbors. Global blurring replaces mesh points with the intersection of polynomial curves generated from points with the same p and q values.

```
dg=blur(dg,'global',order);
```

The optional third input defines the polynomial order (default is 1).

The `extend` method adds new isomesh points to extend the boundaries by one increment in the p and q directions.

```
dg=extend(dg);
```

Linear extrapolation is used to project away from, and then along, existing isopoint groups. This method may be used to expand distortion grid measurements beyond the outermost isopoints, allowing the corrected image to span the full range of the distorted image.

6.3 Saving and recovering image corrections

The `StepWedge` and `DistortionGrid` classes fully support the SDA file format. Objects from either class may be stored in a *.sda file using the `writeFile` function. For example:

```
writeFile(filename,'Step wedge',sw);
writeFile(filename,'Distortion grid',dg);
```

stores the image correction objects described in this chapter to common archive file. See Chapter 2 for more information about this function and the `FileAccess` package.

Archived image correction objects may be pulled from a SDA file using the `readFile` function.

```
previous_sw=readFile(filename,'sda','Step wedge');
```

The restored object is a complete replica of the original source, including measured images and any supplemental data (*e.g.* isopoint selections).

Users are strongly encouraged to save image correction objects to SDA files, particularly when a significant number amount of manual interactions are involved. The classes described in this chapter to not provide export methods! This deliberate choice reflects the integral role of supplemental user selections with the measured image; these images are relatively useless without such information.

Chapter 7

Graphics

The `SMASH` package supports are variety of advanced MATLAB graphics. This chapter highlights some of these features in the `Graphics` and `MUI` (manual user interface) sub-packages. Section 7.1 describes plotting utilities, while Section 7.2 discusses dialog boxes. While these are not strictly distinct features in MATLAB, it is useful to separate the two concepts. Plots are graphical displays of numerical information: lines, images, and so forth. Dialog boxes are graphical control interfaces: buttons, edit boxes, and so forth. Plot figures almost always resizable, whereas dialog boxes usually are not. Section 7.3 discuses the `GraphicOptions` class, which is part of the `Graphics` package. This class is used by other classes, such as `Signal` and `Image`, to control visualization.

7.1 Plot utilities

Figure 7.1 illustrates an example of MATLAB's plot hierarchy. Figures may contain one or more axes objects, and each axes object may contain one or more plotting elements (lines, images, etc.). This section discusses special figures, axes, and plot elements provided by the toolbox that fall within this hierarchy.

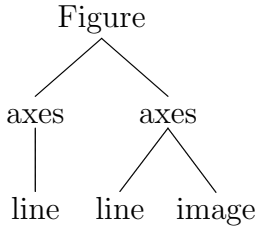

Figure 7.1. Example MATLAB graphics hierarchy

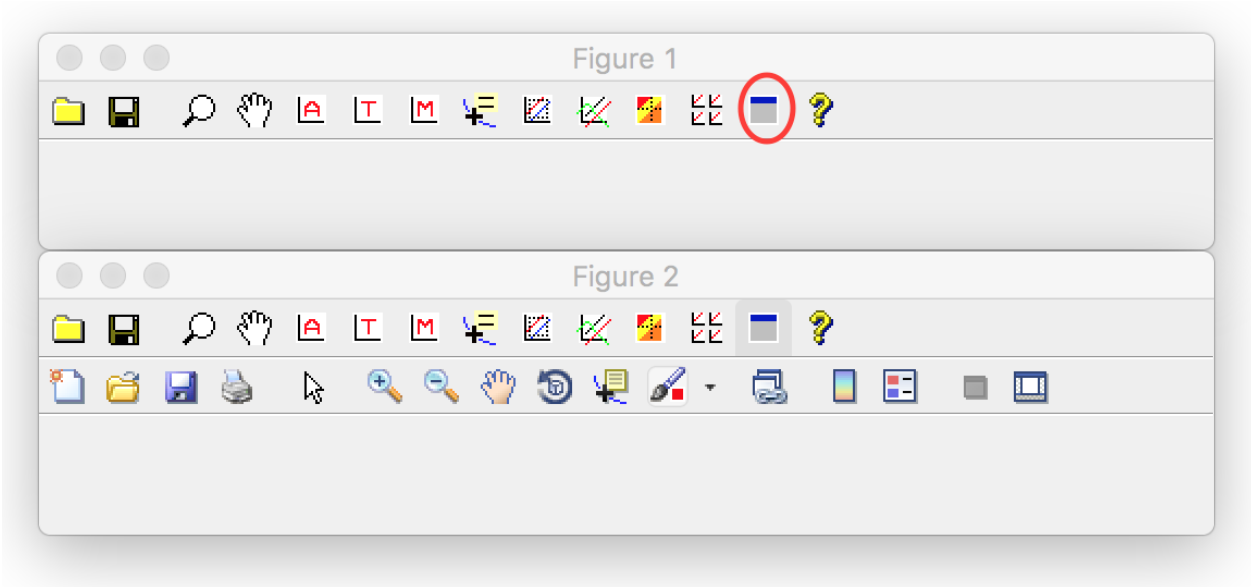

Figure 7.2. Custom figures generated by the `Figure` class. The upper figure is the default state, which hides the standard MATLAB toolbar. The lower figure shows the standard toolbar, made visible after pressing the circled toggle button.

7.1.1 Special figures

The `MUI` package provides the `Figure` class for generic interfaces.

```
object=Figure(...);
```

Creating a `Figure` object generates an empty figure with a custom toolbar as shown in Figure 7.2. By default, this figure differs from a standard figure in several ways.

- The figure has no menus.
- The figure cannot be docked with the command window.
- The standard tool bar is disabled and a custom toolbar is displayed in its place. The toolbar has icons (from left to right) for:
  - Changing the current directory.
  - Saving the figure to a file.
  - Axes zoom.
  - Axes panning.
  - Auto axes scaling.

- Tight axes scaling.
- Manual axes scaling.
- Generating data cursors.
- Calculating region of interest statistics.
- Overlaying data on existing figures.
- Generating horizontal/vertical image slices.
- Cloning axes to a separate figure.
- Showing/hiding the standard toolbar.
- Documentation.

All standard arguments for the `figure` function can be passed to the `Figure` class, though some (notably MenuBar, Toolbar, and DockControls) are ignored at object creation. Once created, figure properties can be modified as needed.

The `Graphics` package provides the `AIPfigure` for publication graphics. These figures match the 8.5 cm column width used by American Institute of Physics journals.

```
fig=AIPfigure(1,height); % one-column figure
fig=AIPfigure(2,height); % two-column figure
```

The optional "height" input accepts strings containing a number and unit abbreviation, such as `'20cm'` or `'2in'`. The default figure height is 10 cm.

7.1.2 Exporting figures

The `printFigure` function prints MATLAB figures to standard graphic file formats. The function is always called with a file name.

```
printFigure(filename,...); % print current figure
printFigure(fig,filename,...); % print specified figure
```

The file extension dictates the graphic format. Vector formats (*.eps and *.pdf) are recommended for figures dominated by lines and text; bitmap formats (*.jpg, *.png, and *.tif) are useful for very large images. In general, *.pdf is usually the best choice.

The `clipFigure` function copies a MATLAB figure to the clip board

```
clipFigure % copy current figure
clipFigure(fig); % copy specified figure
```

Copied figures can be directly pasted into other applications.

```
object=AxesInset(gca);
object.XBound=[0.45 0.55];
object.YBound=[0.8 1.2];
object.Position=[0.10 0.80 0.80 0.15];
update(object);
```

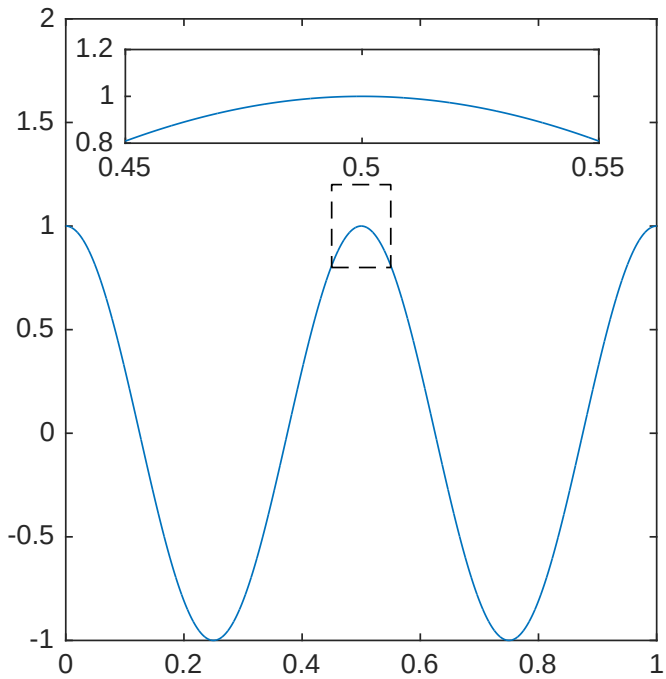

Figure 7.3. Axes inset example

7.1.3 Special axes

The `Graphics` package has a function for generating axes insets.

```
object=AxesInset(target,...);
```

These insets replicate the contents of a source axes to highlight information in a particular region. Figure 7.3 illustrates an example inset built around a local maximum. A dashed rectangle indicates the inset region on the source axes; the inset axes is entirely dedicate to this region. Inset boundaries and positions can be explicitly specified or interactively changed.

The `Graphics` package also has a function for generating axes arrays.

```
ha=AxesArray(rows,columns);
```

```
ha=AxesArray(3,1);
set(ha(2),'YAxisLocation','right');
xlabel(ha(3),'X');
for n=1:3
    ylabel(ha(n),'Y');
end
```

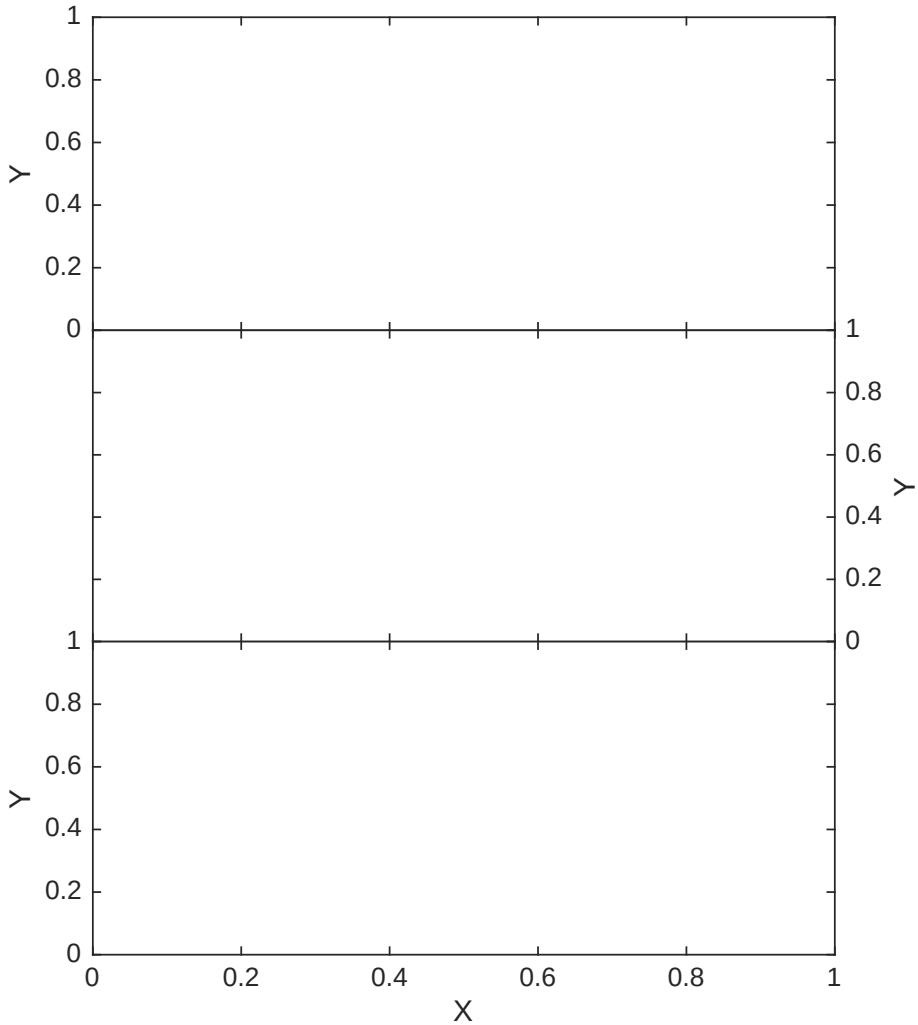

Figure 7.4. Axes array (3×1) where three vertically stack plots use a common horizontal range.

The axes created by this function are tightly packed: there is no space between adjacent figures. Figure 7.4 shows an example of a 3×1 axes array; for clarity, the vertical axis of the center axes is moved to the right side. This type of display is meant for plots sharing common horizontal/vertical ranges. Common vertical ranges are automatically enforced for each axes row; common horizontal ranges are enforced along each axes column.

7.1.4 Special plot elements

The `Graphics` package provides a function for generating two-dimensional error bars.

```
errorbar2(x,y,dx,dy);
```

Error bars are drown as black lines passing through the specified (x, y) points. Widths (dx) and heights (dy) may be specified as scalars are arrays (consistent size required) to indicate common or individual values, respectively. Error bars are drawn without line markers—if specified, markers will appear at every intersection of the error bar. Data markers must be generated separately as shown below.

```
plot(x,y,'ko'); % data points shown as red circles
errorbar2(x,y,dx,dy,[],[],'Color','r'); % red error bars to match
```

The `Graphics` package also provides a function that generates lines with varying color.

```
cline(x,y,color); % 2D line
cline(x,y,z,color); % 3D line
```

The input "color" indicates the color index for each (x, y) or (x, y, z) data point; this array should be the same size as the first 2–3 inputs. The line's actual color is derived from the figure's color map and can be changed after the line is drawn.

```
colormap(jet(64)); % 64 levels from blue to red
```

Local line color is interpolated from levels in the color map.

Figure 7.5 shows an example plot with error bars and a variable color line. The color bar (generated with MATLAB's `colorbar` function) illustrates the link between line color and a parametric variable for that line. Both functions operate in a similar fashion as MATLAB's `line` function: items are added to the current axes without clearing any existing contents. Users should be aware that this has several implications.

- It is not necessary to "hold" an axes when drawing error bars and variable color lines.
- Axes limits are not automatically updated when these elements are drawn. It may be necessary to reset the limits:

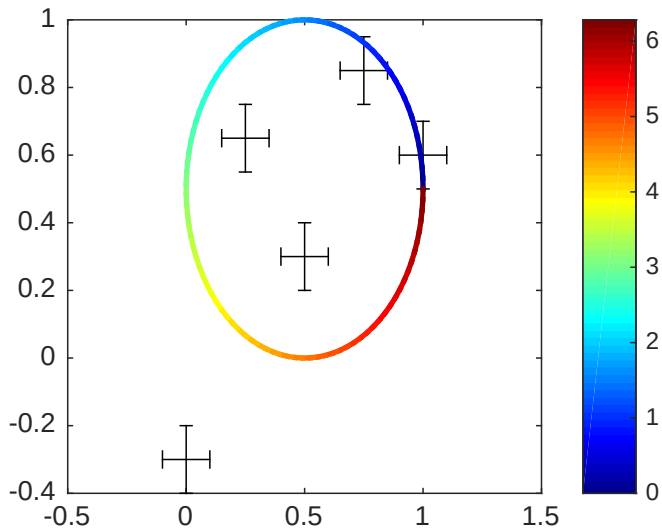

Figure 7.5. Example plot with error bars and a line with varying color

```
axis auto
```

to ensure that newly added items are visible.

- Axes rotation is not changed when generating a three-dimensional lines with varying color. As such, the line may appear two-dimensional because the default view is directly along the $-z$ axis. Updating the view angle:

```
view(3);
```

reveals three-dimensional behavior (where it exists).

7.2 Custom dialog boxes

MATLAB provides a number of standard dialog boxes to prompt users for information. Unfortunately, most of these dialogs are based around a single type of interaction: pressing a button, entering data, or selecting an item (or items) from a list. Prompting users for different things at the same time is not possible with the standard dialogs, and building new dialog boxes from scratch is extremely tedious.

The `MUI` package provides a `Dialog` class that considerably simplifies custom dialog creation. Rather than positioning individual controls, developers can specify the type of control

and focus on how the controls operate. Dialog construction requires some advanced MATLAB knowledge, particularly the use of callback functions, but this class allows users to concentrate on what their dialog boxes should do rather than how they look.

7.2.1 Creating dialogs

Custom dialog box construction begins with the creation of a `Dialog` object. Dialog objects are associated with a MATLAB figure, and deleting either one of these items automatically deletes the other. The `Dialog` class is derived from MATLAB's `handle` class, which operates differently than data classes used throughout most of the SMASH toolbox. Copied handle objects are not independent, and any change made to one object is automatically transferred to all copies of that object. Users unfamiliar with this concept should refer the MATLAB documentation (`doc handle`) to learn more.

Dialog boxes have a number of editable properties.

Name controls associated figure's title bar (text).

Locked controls the ability (or inability) for new blocks to be added to the figure (logical).

Modal controls the figure's window style (normal or modal).

Hidden controls the figure's visibility.

Properties changes are applied immediately after the object is altered.

For best results, Dialog objects should be created as shown below.

```
object=Dialog();
object.Hidden=true;
object.Name='My dialog box';
% add control blocks
locate(object,...);
object.Hidden=false;
object.Modal=true; % modal dialogs only
```

Hiding the dialog allows faster figure rendering, so end user sees only the finished product. Dialogs can be placed at a particular screen location or located with respect to another figure.

7.2.2 Control blocks

Control blocks group one or more related elements in a dialog box. Blocks are added to the dialog using the `addblock` method.

```
h1=addblock(object,'edit','Input something:');
h2=addblock(object,'medit','Write several things:',40,5);
choices={'Choice A','Choice B','Choice C'};
h3=addblock(object,'popup','Make a choice:',choices);
h4=addblock(object,'edit_button',{'Another input' 'Select'});
h5=addblock(object,'table',{'Column1' 'Column2' 'Column2'},[],5);
h6=addblock(object,'button',{' OK ',' Apply ',' Cancel '});
```

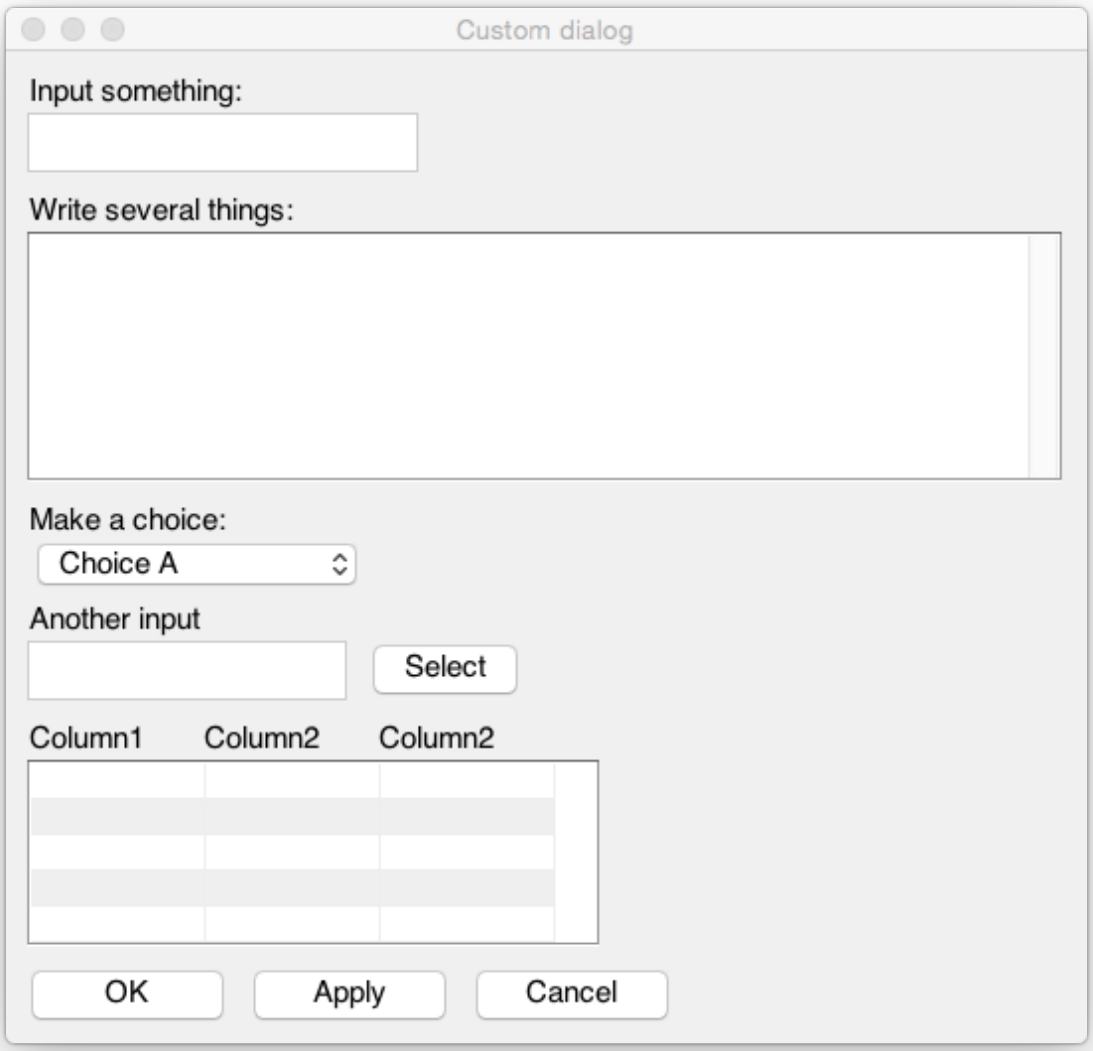

Figure 7.6. Custom dialog example

```
h=addblock(object,style,...);
```

Supported block styles are summarized below.

text blocks are a single line of non-editable text. They are automatically sized to fit the input "value" or an optional minimum width (in characters).

```
h=addblock{object,'text',value,minwidth);
```

edit blocks are a single line of editable text with a static text label above.

```
h=addblock{object,'edit',label,minwidth);
```

The edit box is initially empty and is sized by the input "label" and the optional minimum width.

medit blocks are multiple line of editable text with a static text label above.

```
h=addblock{object,'edit',label,minwidth,height);
```

The multi-line edit block is initially empty.

button is a horizontal collection of push buttons.

```
h=addblock(object,'button',name,minwidth)
```

The input "name" is a cell array of labels for each button. Buttons are automatically sized by the widest label or the requested minimum width.

toggle is a horizontal collection of toggle buttons.

```
h=addblock(object,'toggle',name,minwidth)
```

This block operates in a similar fashion as a button block.

check is a horizontal collection of check boxes.

```
h=addblock(object,'check',name,minwidth)
```

This block operates in a similar fashion as a button block.

radio is a horizontal collection of radio buttons.

```
h=addblock(object,'radio',name,minwidth)
```

This block operates in a similar fashion as a button block.

listbox is a list box (one ore more lines) with a text label above.

```
addblock(object,'listbox',label,choices,mindwidth);
```

The input "label" is placed about the listbox. The input "choices" is a cell array that defines the box contents.

popup is a single-line popup menu with a text label above.

```
h=addblock(object,'popup',label,choices,minwidth);
```

This block operates in a similar fashion as the listbox.

edit_button is similar to an edit block with a push button to the right of the text box.

```
addblock(object,'edit_button',label,minwidth)
```

The input "label" is a cell array the defines the label above the edit box and the button on the right.

popup_button is similar to an popup block with a push button to the right of the popup menu.

```
addblock(object,'popup_button',label,minwidth)
```

table is a uitable that uses separate text box as column labels.

```
h=addblock(object,'table',label,minwidth,rows)
```

The input "label" is a cell array of labels for each table column; "minwidth" defines the minimum width of each column. The input "rows" controls the visible height of the table. If the table is assigned more information that the number of rows, a scroll bar appears for vertical navigation.

Details of each block style are available in the online documentation.

Figure 7.6 shows how these control blocks appear in OS X. Control display will vary across different operation systems, but relative size and positioning will be the same.

7.2.3 Refining and using dialogs

Whenever a control block is added to a dialog box, graphic handles for each item in the block are returned as outputs of the `addblock` method. These handles can be used to modify each graphic. Developers may change any of the block graphic properties, but revisions are usually limited to the String and Callback properties. The latter is particularly important for making a dialog box active.

Three handle arrays—`h1`, `h2`, and `h3`—were created in the previous example. The first two arrays contain two handles: the text object and the second is the edit/popup object; the third array contains three button handles. Each handle can be assigned its own callback function. For example:

```
set(h3(1),'Callback',@closeDialog);
function closeDialog(varargin)
       delete(gcbf);
end
```

deletes the dialog box (as well as the figure) when the user presses the "OK" button. Refer to MATLAB's documentation for more information about creating callbacks and passing information to those callback.

The `Dialog` class contains a useful method for extracting data from a dialog box.

```
data=probe(object);
```

The output "data" is a cell array of values from the editable parts of each control block (in the order they were created. If called for the dialog box shown in Figure 7.6, the cell array "data" would have two fields: ' ' (from the empty edit box) and 'Choice A' (from the popup menu). This method can be called whenever a `Dialog` object is available, including callback functions and the command window.

7.3 Object graphic options

Several classes (`Signal` and `Image`) in the SMASH toolbox use a common framework for managing graphic options in their `view` method. A `GraphicOptions` object, kept as editable property in these classes, manages these options and prevents users from specifying invalid settings. The `GraphicOptions` class is part of the `Graphics` package.

A current list of `GraphicOptions` properties is given below.

AspectRatio controls axes aspect ratio. The default value is `'auto'`, but users request `'equal'` to equally scale horizontal and vertical directions .

AxesColor controls the axes control (default is ' `w'`).

Box shows the axes as open (' `off'`) or closed (`'on'`).

ColorMap specifies the figure's color map. The default value is equal to the output `'jet(64)'`, but any three-column table (red green blue) can be specified.

FigureColor controls the figure color.

LineColor controls line color.

LineStyle controls line style.

LineWidth controls line width (in points).

Marker controls marker type.

MarkerSize controls marker size (in points).

MarkerStyle controls marker style (`'open'` or `'closed'`).

PanelColor controls uipanel color.

Title controls axes title.

XDir controls x-axis direction (' `normal'` or `'reverse'`).

YDir controls y-axis direction (`'normal'` or `'reverse'`).

Actual implementation of these settings vary with class—not all properties are used in every visualization.

Note that graphic options are only used when an object's `view` method is called. For example:

```
object.GraphicOptions.LineColor='r';
view(object);
object.GraphicOptions.Linecolor='b';
```

changes the line color setting to red and then blue, but plotted line is red. Changing settings after visualization does not alter existing plots.

This page intentionally left blank.

Chapter 8

Programs

This chapter discusses programs that a distributed with the SMASH toolbox. MATLAB does not formally distinguish programs from any other function, but in this context, programs are functions with a self-contained interface (usually graphical) that address a specific task. Programs are launched by name, deliberating hiding their underlying complexity (including dependencies on SMASH toolbox packages) from the end user.

Programs are located in a sub-directory of the SMASH toolbox, which by default is *not* on the MATLAB path. The utility function `loadSMASH` places programs on the path so they be used.

```
loadSMASH -program * % load all programs
PointVISAR % launch PointVISAR
...
```

Routinely used programs should be loaded at the beginning of each MATLAB session through the use of a startup file. NOTE: programs should not be manually added to the path as the internal organization of the SMASH toolbox may change! The `loadSMASH` utility will always remain consistent with the SMASH toolbox.

Three types of program are available in the SMASH toolbox. Section 8.1 discusses data analysis programs. Section 8.2 summarizes data synthesis programs. Section 8.3 describes file processing programs

8.1 Data analysis

The SMASH toolbox includes three data analysis programs. These programs reduce measured signals to time-resolved velocity. The oldest of these, PointVISAR, supports VISAR (Velocity Interferometer System for Any Reflector)[8] measurements. PDV (Photonic Doppler Velocimetry)[9] measurements are managed by THRIVE and SIRHEN.

8.1.1 PointVISAR

The PointVISAR program reduces conventional (3 signals) and push-pull (2 or 4 signals) VISAR measurements to time-resolved velocity. A comprehensive discussion of this reduction may be found in Reference 10. A brief summary of the process is given below.

- Measured signals (2–4, depending on the VISAR configuration) are balanced, time corrected, and smoothed as necessary.
- A pair of quadrature signals ($D_x(t)$ and $D_y(t)$) are derived from the processed signals.
- The fringe shift $F(t)$ is calculated from the quadrature signals. Ideally:

$$\tan\left[2\pi F(t)\right] = \frac{D_y(t)}{D_x(t)} \tag{8.1}$$

  but imperfect quadrature can be incorporated into the analysis.
- The apparent velocity $v^*(t)$ is calculated from the VISAR approximation:

$$v^*(t) \approx KF(t) \tag{8.2}$$

  where K is the fringe constant for the VISAR system.
- Fringe addition/subtraction is used (as needed) to reconcile measurements of the same velocity with different fringe constants.

Multiple VISAR records can be loaded into the program at the same time.

Figure 8.1 shows the main screen of the PointVISAR program. The left portion of the screen displays the quadrature signals, a quadrature plot, and the result for the active VISAR record; the right portion shows results for all selected VISAR records. Result plots display velocity, fringe shift, or VISAR controls; right-clicking on the y-axis allows user to select between these result types. Buttons labelled "choose" allow users to select the active record and determine which record results are displayed.

The Record menu allows new records to be added and existing records to be modified. When new records are added, users are prompted to select the VISAR configuration and the measurement file(s). After measurement is loaded into to memory, a record settings screen (Figure 8.2) shows plots of the raw, processed, and quadrature signals. Users can adjust signal shifting/scaling/smoothing as needed and define system parameters (fringe constant, quadrature parameters, etc.). An optional (but highly recommended) label can be set in this window to distinguish this record from others in the main program window. Pressing "Done" in the Record menu closes the record menu and returns the program to the main screen.

Fringe addition/subtraction and global time shifting is applied in the program's main screen (Processing menu) to allow simultaneous comparison between VISAR records. When

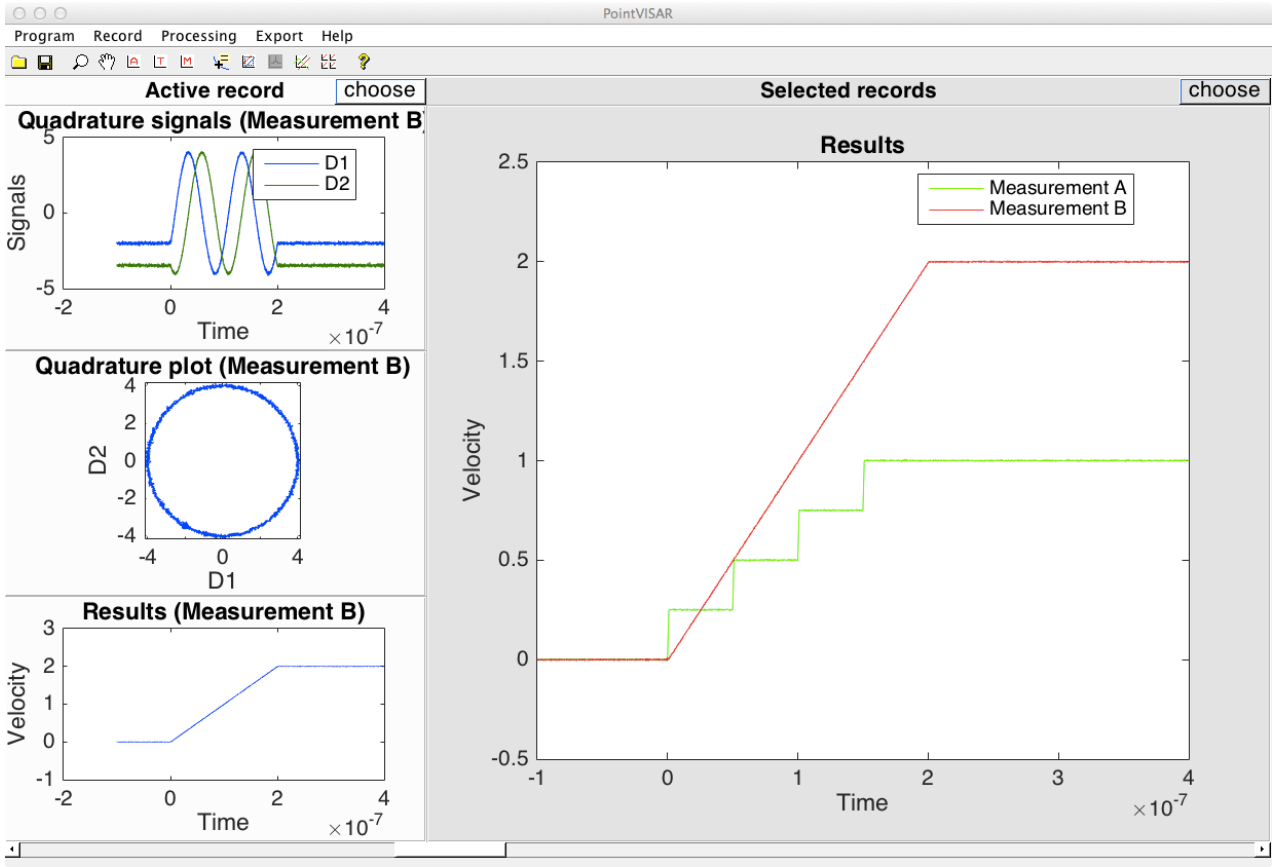

Figure 8.1. PointVISAR main screen

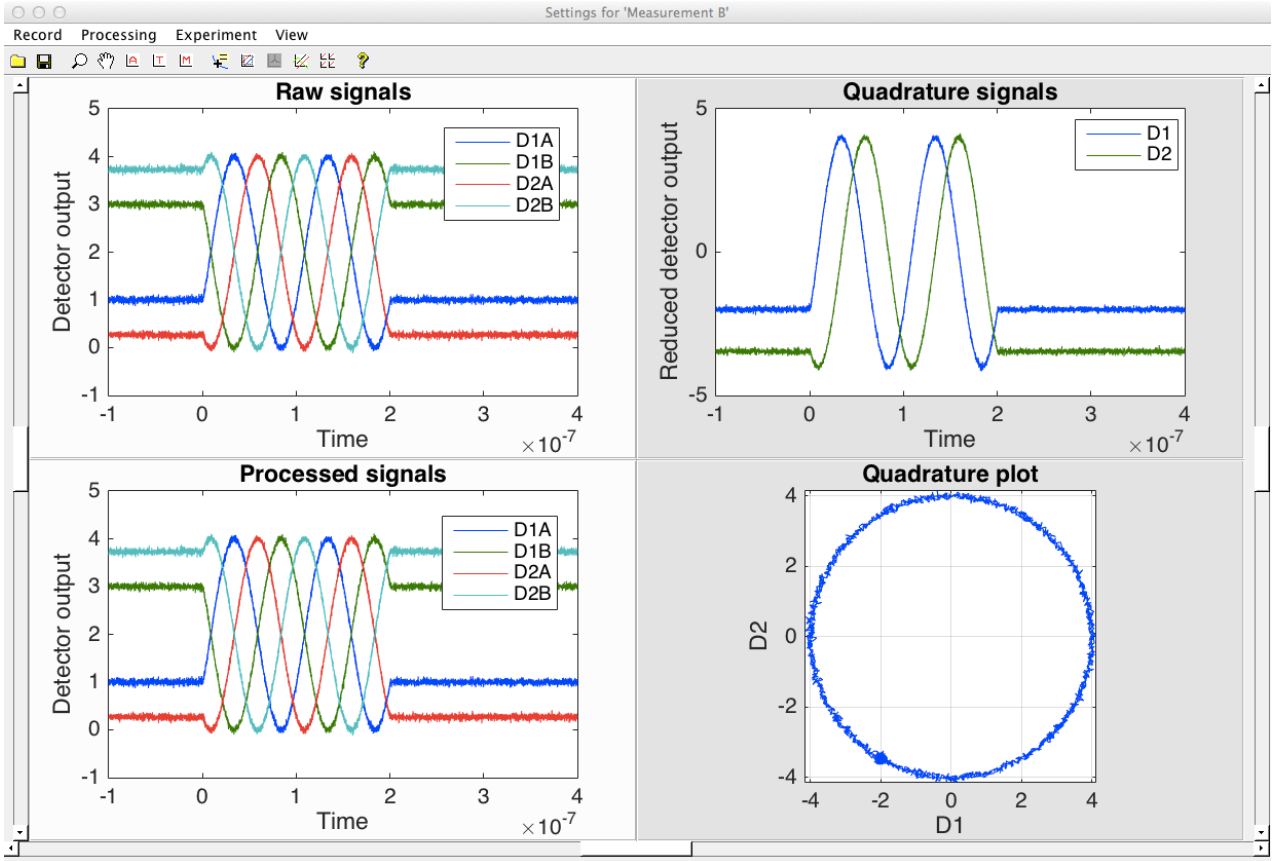

Figure 8.2. PointVISAR record settings screen

analysis is complete, VISAR results may be exported to separate text files (one record per file). Analysis configurations may also be saved to a text file, storing all settings for future PointVISAR sessions.

8.1.2 THRIVE

The THRIVE (THRee Interferometer VElocimetry) program processes quadrature PDV measurements, sometimes referred to as "triature". Reference 11 completely describes the theory of quadrature PDV analysis and the use of THRIVE for this analysis. While the construction of VISAR and quadrature PDV systems are very different, THRIVE is conceptually similar to PointVISAR in many ways. Key differences are noted below.

- THRIVE constructs quadrature signals from measurements that are nominally 120° out of phase with one another (instead of 90/180°).
- Fringe shifts derived in quadrature PDV are proportional to apparent displacement instead of velocity. THRIVE uses smoothed numerical differentiation to determine velocity.

8.1.3 SIRHEN

The SIRHEN (Sandia InfraRed HEterodyne aNalysis) program reduces conventional and frequency-shifted PDV measurements. Reference 12 provides a complete description of the program and its underlying principles. SIRHEN processes individual PDV signals, operating purely on frequency-domain analysis instead of phase methods (as in PointVISAR and THRIVE).

8.2 Data synthesis

Two data synthesis programs—fringen and pyrosim—are included with the SMASH toolbox. These programs model the behavior of hypothetical measurements, which is helpful in design of new diagnostic systems and data reduction algorithms.

8.2.1 fringen

The fringen program generates fringes for hypothetical velocimetry measurements. Given a velocity $v(t)$, the program synthesizes VISAR or PDV signals for a various system configurations (wavelength, light level/type, etc.). Signal imperfections, such as noise, can also be

simulated in the program. Signals generated by fringe follow the exact physical relationships for VISAR and PDV measurements—no approximations are used.

Reference 13 completely describes the fringen program. In most cases, fringen is used in conjunction with a reduction program (such as SIRHEN) for testing different analysis methods.

8.2.2 pyrosim

The pyrosim program simulates the operation of a pyrometer. Given a set of measurement conditions (collection diameter/angle, emissivity, wavelength range, etc.), the program calculates the power and photon collection rate of the sensor as a function of temperature. Given the sensitivity of the sensor, the program can also estimate output voltage. Reference 14 describes the operating theory and use of the program.

8.3 File processing

The SMASH toolbox includes three file processing programs. The analyzeSinusoid program processes digitizer data files, while the convertLUNA program processes optical backscatter reflectometer scans. The dataninja program extracts (x, y) data from a bitmap graphic file.

8.3.1 analyzeSinusoid

The analyzeSinusoid program processes digitizer signals as noisy sinusoids:

$$s(t) = A\cos(2\pi f_0 t + \delta) + \sigma R(t) \tag{8.3}$$

to determine three parameters of interest. Signal amplitude A represents the characteristic strength of the signal's harmonic content (at frequency f_0), while root-mean-square noise σ summarizes the signal's random content. These quantities are cast in the time domain for conceptual clarity, but in practice they are calculated in the frequency domain, using a calibrated approach to account for the digitizer's time base. For example, signal amplitude is determined from a comparison of spectral peak amplitude with a spectral peak amplitude for a perfect sinusoid with $A = 1$ over the same time base.

Figure 8.3 shows an example screen from the analyzeSinusoid program. The digitizer file in this example contains four signals (one for each channel), each summarized on a separate row. The strongest frequency in each signal is listed in the second column, along with the corresponding amplitude (in the third row). These calculations use the entire spectral domain of the signal (DC to the Nyquist limit) by the default, but analysis can be restricted

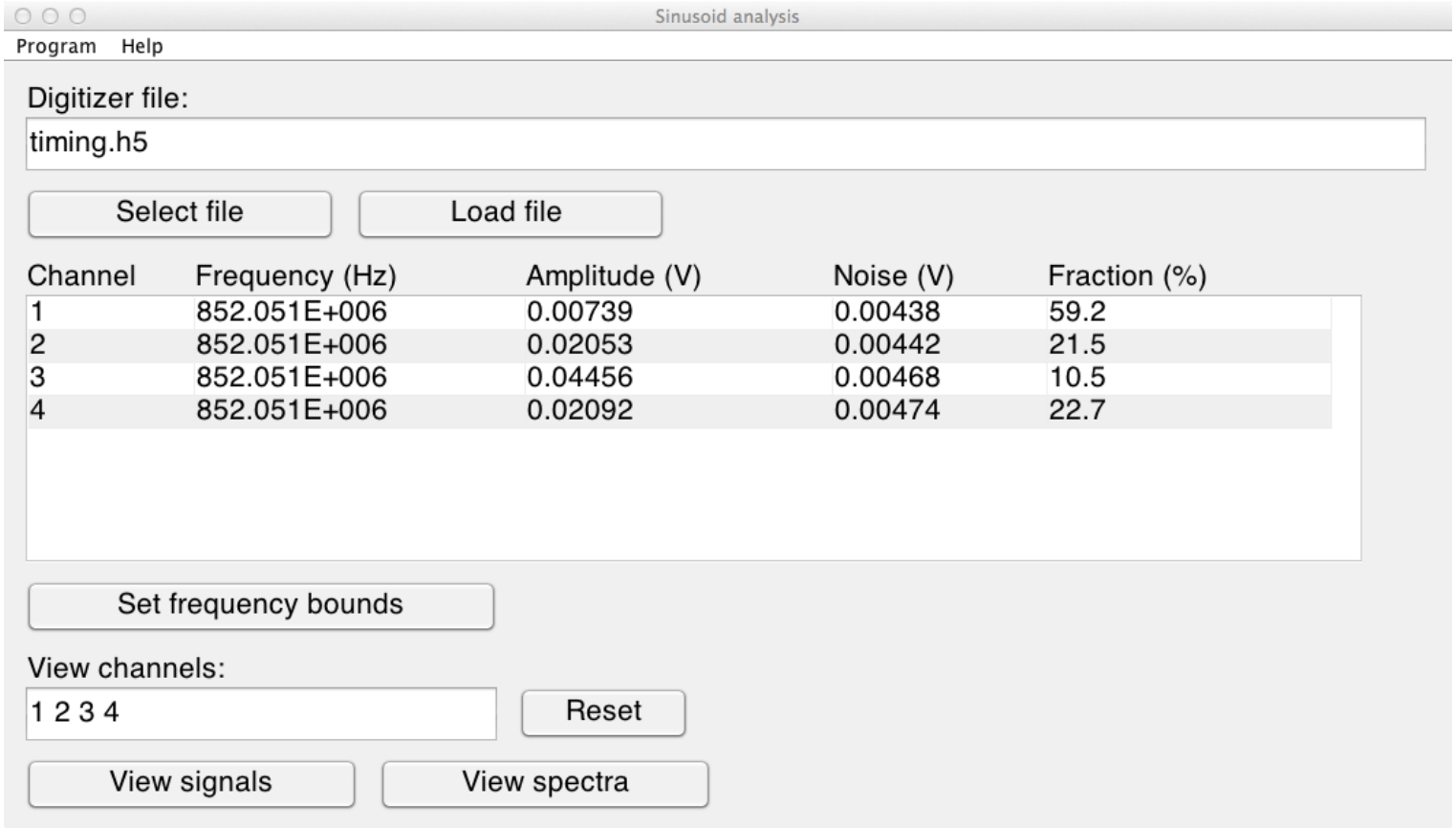

Figure 8.3. The analyzeSinusoid program

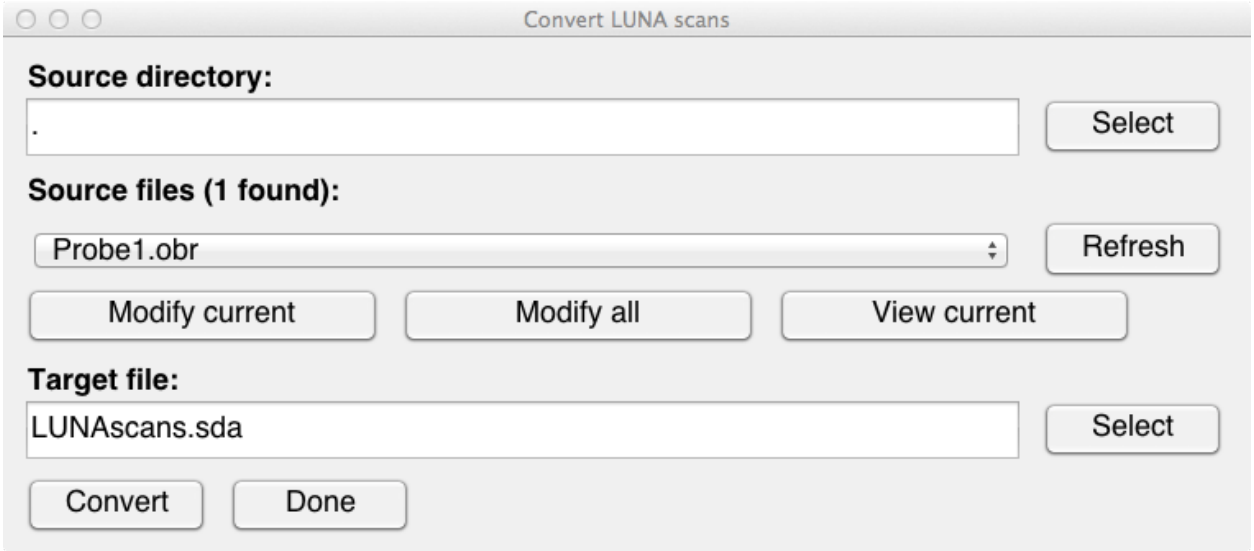

Figure 8.4. convertLUNA program

to a particular domain by setting frequency bounds (button underneath the table). Signals and spectrograms for each digitizer channel can be displayed (bottom buttons) in separate figures; visualization can use some or all of the channels in the digitizer file.

This function provides basic characterization of harmonic signals. It's primary purpose is to aid the configuration and testing of frequency-shifted PDV measurements. Frequency estimates are useful for tuning laser wavelengths, and the amplitude helps to define the appropriate vertical range(s) for the digitizer. The noise fraction (σ/A) plays an important role in the limiting performance of a PDV measurement.[15]

8.3.2 convertLUNA

The convertLUNA program processes individual scans (*.obr files) from a LUNA backscatter reflectometer into a Sandia Data Archive (*.sda file). This conversion involves considerable data compression: individual scans starting around 130 megabytes can be represented by less than megabtye with considerable fidelity.

Figure 8.4 shows an example screen from the convertLUNA program. The program reads *.obr files in the specified source directory and writes each as a separate record (named after the the source file) in the specified archive file. These records are `LUNA` class objects, which are defined in the `Velocimetry` package (to be described in a future report). These objects store the linear amplitude (units of inverse nanoseconds) measured by the LUNA as a function of time (in nanoseconds). The class provides methods for visualization and analysis (locating peaks and determining return loss).

Several types of data compression are used in converted LUNA scans. Thrifty data representation is always present: redundant information is omitted, large arrays use single precision instead of double precision, and so forth. User-defined modifications also play an important role. LUNA scans are sampled at less than 0.001 ns, when for most practical purposes 0.01 ns (or larger) intervals are sufficient; scans can also be cropped to region of interest to reduce file size even further. The "Modify current" and "Modify all" buttons in Figure 8.4 allow users to individually or globally modify LUNA scans in the source directory. Modifications are specified as a time step and a time range, both in units of nanoseconds. Time steps are always a single number, while the time range is pair of numbers (start and stop); `-Inf` and `+Inf` can be used as place holders when needed.

The current source file in convertLUNA is selected with the popup menu. Pressing the refresh button to the right of this menu re-reads all *.obr files in the source directory, eliminating any time step/range modifications. The "View current" button displays the scan (with any modifications) in a separate figure.

8.3.3 datninja

The datninja program performs digitization of two-dimensional plots stored in a bit-mapped image file (*.jpg, *.png, etc.). Figure 8.5 shows a screen shot of the program using a rotated test image. The entire image is shown on the left side of the figure; to maximum visibility, images should be cropped as closely to the plot of interest as possible. Inside the full image is a dashed box that defines the zoom region. The contents of this region are displayed on the left side of the figure. The size of the zoom region can be adjusted (by factors of two) using the "zoom in" and "zoom out" buttons. The zoom region can be moved by clicking on the full image or by pressing the arrow keys.

Digitizing data from an image is a multi-step process.

1. The image is rotated (90° increments) and/or flipped for general orientation using the Tools menu (as needed).

2. Axis scaling (linear or logarithmic) is selected in the Options menu (as needed).

3. A set of reference points are selected. Reference points are selected by clicking in the zoom region at locations where the (x, y) coordinate is known. A minimum of three reference points (not on a straight line) must be selected; additional points are used to refine the transformation from pixel location (m, n) to coordinates (x, y).

4. Data points of interest are selected by clicking in the zoom region. Radio buttons allow quick transition between reference and data point selection.

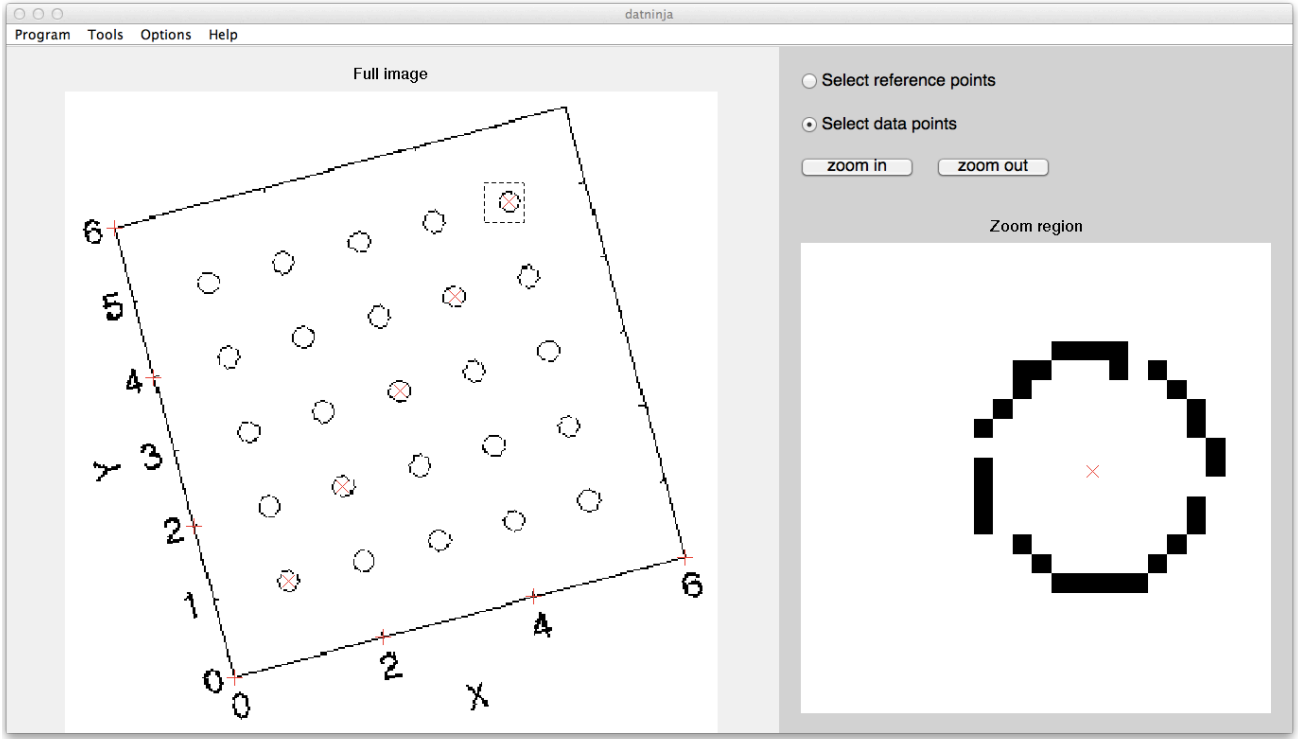

Figure 8.5. Screen shot `datninja`. Reference points are shown as red plus signs; data points are shown as red crosses. NOTE: the low resolution and rotated coordinate system are intentionally used in this example to mimic a poorly scanned figure.

```
datninja points saved 11-Jan-2016 21:46:28
Data format: m n x y
Image file: exampleB.tif
reference: 30.71 83.09 0 6
reference: 54.22 173.29 0 4
reference: 78.41 262.77 0 2
reference: 374.94 281.59 6 0
reference: 283.43 304.66 4 0
reference: 192.48 329.71 2 0
reference: 103.19 354.19 0 0
data: 268.64 67.23 4.99183 5.0115
data: 235.81 124.24 4.00325 4.01284
data: 202.68 181.26 3.0085 3.01544
data: 168.14 238.94 1.981 2.01237
data: 135.45 295.64 0.997107 1.01927
```

Figure 8.6. Example datninja file

Right-clicking on any selected point allows that point to be individually removed. Reference point locations may also be edited by right-clicking.

The transformation from pixel location to data coordinate is modeled as a linear transformation of complex values.

$$\begin{gathered} \vec{x}_D + i\vec{y}_D = A_D\,(\vec{p} + i\vec{q}) \\ A_D \equiv \left[\begin{array}{ccc} \vec{m}_D & \vec{n}_D & \vec{1} \end{array}\right] \end{gathered} \tag{8.4}$$

A best estimate for the complex parameter vector (six values in all) is made from the selected reference points.

$$\begin{gathered} (\vec{p} + i\vec{q}) \approx A_R \backslash\, (\vec{x}_R + i\vec{y}_R) \\ A_R \equiv \left[\begin{array}{ccc} \vec{m}_R & \vec{n}_R & \vec{1} \end{array}\right] \end{gathered} \tag{8.5}$$

Logarithmic mapping is used as needed if logarithmic scaling is selected for either axis.

All selected points in a datninja session can be saved to a text file (Program menu). Figure 8.6 shows an example file generated from the reference/data selections in Figure 8.5. Pixel values and locations for each selection are printed on separate lines. Saved files can be read back into fringen for additional refinement.

Digitized data stored in a file may be read into MATLAB using the `datninja_reader` function.

```
[x,y]=datninja_reader(filename);
```

This page intentionally left blank.

Chapter 9

Summary and future work

The SMASH toolbox gathers packages, programs, and utilities under a common MATLAB framework. The capabilities of the toolbox range from low-level data processing to self-contained graphical interfaces for data reduction. The report covers the first formal release, version 1.0.

9.1 Version 1.0 summary

This release provides several core packages.

- `FileAccess` for reading data from and writing data to files.
- `SignalAnalysis` for processing one-dimensional information.
- `ImageAnalysis` for processing two-dimensional information.

Capabilities are highly integrated between these packages. For example, `Signal` objects are loaded by functions from the `FileAccess` package, and slicing an `Image` object produces a `SignalGroup` object. These packages also contain support classes—such as `ShortTime` and `StepWedge`—for expanded capability.

The toolbox contains various support packages.

- `Graphics` for custom visualization.
- `MUI` for manual user interfaces.
- `General` and `System` for various low-level features.

These packages are intended for developers and advanced users.

The SMASH toolbox also includes programs for:

- Velocity interferometer analysis (PointVISAR, THRIVE, SIRHEN).

- Data synthesis (fringen and pyrosim).
- File processing (analyzeSinusoid, converLUNA, and datninja).

Many of these programs have detailed user manuals separate from this report.

9.2 Future work

By design, the toolbox will continue to evolve based on the needs of its user base. Planned work for future releases is outlined below.

9.2.1 Packages under development

The next formal toolbox release, version 1.1, is tentatively scheduled for January 2017. The following packages may be part of this release.

- `MonteCarlo` (uncertainty analysis)
- `ROI` (Region Of Interest selection)
- `CurveFit`
- `PDE` (Partial Differential Equation solvers)
- `NTOF` (Neutron Time Of Flight)
- `Radiograph` (x-ray imaging)
- `DynamicMaterials` (equation of state calculations, impact calculator, etc.)

Some of these package may be postponed to later versions, based on their level of readiness, and additional packages may be considered.

Packages within the SMASH toolbox will likely to evolve after their formal release. These changes will usually be limited to new capabilities and bug fixes. Existing features provided by the toolbox should appear as unchanged as possible to the standard user.

9.2.2 Future programs

A new round of velocimetry analysis programs, notably PointVISAR and SIRHEN, will be incorporated into the SMASH toolbox. In addition to new analysis capabilities, these programs will built on graphical capabilities from the `MUI` package. This change should enhance the user experience while reducing development time.

References

[1] MATLAB software, The Mathworks Inc., Massachusetts, United States.

[2] D.H. Dolan. Portable File Format (PFF) specifications. Technical Report SAND2015-0577, Sandia National Laboratories, (2015).

[3] L.P. Mix, R.S. Coats, and D.B. Seidel. PFIDL: procedures for the analysis and visualization of data arrays. In *Tri-Laboratory Engineering Conference on Computational Modeling*, (1995).

[4] D.H. Dolan and T. Ao. Sandia Data Archive (SDA) file specifications. Technical Report SAND2015-1118,, Sandia National Laboratories, (2015).

[5] The HDF Group. Hierarchical data format, version 5, (1997–2014). `http://www.hdfgroup.org/HDF5/`.

[6] A. Savitzky and M.J.E. Golay. Smoothing and differentiation of data by simplified least squares procedures. *Analytical Chemistry* **36**, 1627 (1964).

[7] F.J. Harris. On the use of windows for harmonic analysis with the discrete Fourier transform. *Proceedings of the IEEE* **66**(1), 51 (1978).

[8] L.M. Barker and R.E. Hollenbach. Laser interferometer for measuring high velocities of any reflecting surface. *Journal of Applied Physics* **43**, 4669–4675 (1972).

[9] O.T. Strand, D.R. Goosman, C. Martinez, T.L. Whitworth, and W.W. Kuhlow. Compact system for high-speed velocimetry using heterodyne techniques. *Review of Scientific Instruments* **77**, 83108 (2006).

[10] D.H. Dolan. Foundations of VISAR analysis. Technical Report SAND2006-1950, Sandia National Laboratories, (2006).

[11] D.H. Dolan and S.C. Jones. THRIVE: a data reduction program for three-phase PDV/PDI and VISAR measurements. Technical Report SAND2008-3871, Sandia National Laboratories, (2008).

[12] T. Ao and D.H. Dolan. SIRHEN: a data reduction program for photonic Doppler velocimetry measurements. Technical Report SAND2010-3628, Sandia National Laboratories, (2010).

[13] D.H. Dolan. Velocimetry signal synthesis with fringen. Technical Report SAND2011-0582, Sandia National Laboratories, (2011).

[14] D.H. Dolan. Pyrometry simulator (`pyrosim`) for diagnostic design. Technical Report SAND2011-7778, Sandia National Laboratories, (2011).

[15] D.H. Dolan. Accuracy and precision in photonic Doppler velocimetry. *Review of Scientific Instruments* **81**, 53905 (2010).

DISTRIBUTION:

1	MS 0899	Technical Library, 9536 (electronic copy)

This page intentionally left blank.

Sandia National Laboratories